

PROGRAMMING OF MOBILE TERMINALS

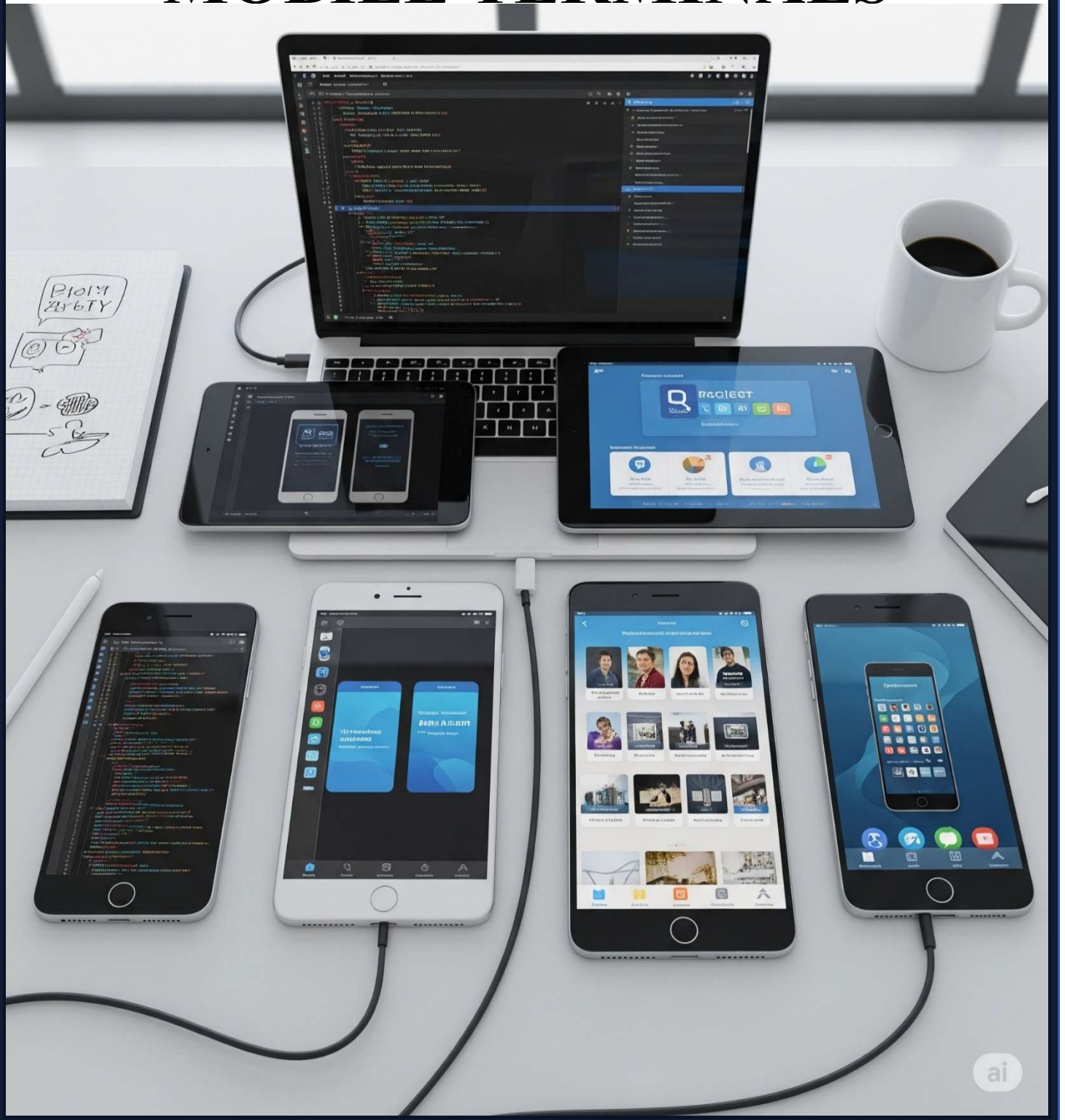


Table of Contents

MODULE 01.....	1
GENERALITIES ON MOBILE TERMINALS.....	1
1.1. INTRODUCTION.....	1
1.1.1. Consumer-Grade Mobile Terminals.....	1
1.1.2. Enterprise and Specialized Mobile Terminals	5
1.2. FEATURES OF MOBILE TERMINALS.....	7
i. Hardware Features	7
ii. Software Features.....	7
iii. Connectivity Features	8
1.2.1. Advantages of Mobile Terminals	8
1.2.2. Disadvantages of Mobile Terminals.....	9
MODULE 02.....	10
INTRODUCTION TO MOBILE OPERATING SYSTEMS	10
2.1. INTRODUCTION.....	10
2.1.1. Core Functions of an Operating System	10
2.1.2. Types of Operating Systems	10
2.1.2.1. Android OS	11
2.1.2.2. Key Features of the Android OS.....	11
2.1.2.3. Advantages and Disadvantages of the Android OS.....	11
2.1.2.4. History and Evolution of the Android OS.....	12
i. The Early Years (2008 - 2011): The Foundations	13
ii. The Tablet Era & Design Overhauls (2011 - 2013).....	15
iii. The Modern Era (2014 - Present).....	17
2.2. THE ANDROID ARCHITECTURE	20
i. The Linux Kernel.....	20
ii. Hardware Abstraction Layer (HAL) & Native Libraries	21
iii. Android Runtime (ART)	21
iv. Application Framework	21
v. Applications	22
2.3. THE iOS ENVIRONMENT	24
2.3.1. Key Features of the First iPhone	25
2.3.2. Key Features of the iOS Environment	29
i. User Interface and Design	29
ii. Ecosystem and Integration	30

iii. Security and Privacy	30
iv. Apple Intelligence.....	30
2.3.3. History and Evolution of the iOS Environment	35
i. The Early Years (2007–2009): A New Paradigm	37
ii. The Renaming and Major Overhauls (2010–2013).....	37
iii. The Modern Era (2014–Present)	38
2.3.4. Advantages and Disadvantages of the iOS Environment	38
2.3.5. Differences between the Android OS and the iOS Environment	39
2.4. THE iOS ARCHITECTURE	40
i. Cocoa Touch Layer – User Interaction & App Behaviour	40
ii. Media Layer – Graphics, Audio, and Video	41
iii. Core Services Layer – Data, Networking, and System Services	41
iv. Core OS Layer – Low-Level Access & Security	42
v. Hardware Layer – The Physical Foundation.....	42
MODULE 03.....	43
INTRODUCTION TO ANDROID APPLICATION PROGRAMMING USING ANDROID STUDIO	43
3.1. INTRODUCTION.....	43
3.1.1. Native Vs Cross-Platform Applications.....	44
3.2. How to Download and Install Android Studio on Windows	46
3.2.1. Key Components of Android Studio.....	47
i. Code Editor.....	47
ii. Layout Editor	47
iii. Android Emulator	48
iv. Gradle Build System	50
3.3. Create Your First Android App	51
3.3.1. Layouts Basics	52
3.3.2. Components or Views Basics.....	62
3.3.2.1. Layouts in Views	65
3.4. Android Application Core Components.....	80
i. Activity	80
ii. Services.....	82
iii. Broadcast Receivers	82
iv. Content Providers	82
3.5. The Kotlin Programming Language	85
3.5.1. Kotlin Syntax Basics	85

3.5.2. Key Features of Kotlin	86
i. Null Safety	86
ii. Conciseness and Reduction of Boilerplate	87
iii. Interoperability with Java (JVM)	87
iv. Extension Functions	87
v. Coroutines	87
vi. Functional Programming Features	88
i. Virtual Machines	88
ii. Libraries and APIs	89
iii. User Interface (UI) Frameworks	89
iv. Development Tools	89
v. Targeting Platforms	89
MODULE 04	90
INTRODUCTION TO JETPACK COMPOSE	90
4.1. INTRODUCTION	90
4.1.1. Composable Functions	90
4.1.2. Layouts	92
4.1.3. Material Design	92
4.1.4. Lists and animations	92
MODULE 05	93
INTRODUCTION TO THE REACT NATIVE UI FRAMEWORK	93
5.1. INTRODUCTION	93

MODULE 01

GENERALITIES ON MOBILE TERMINALS

1.1. INTRODUCTION

A **mobile terminal** is essentially any end-user device designed to connect to a mobile or wireless network, enabling the exchange of **data, voice, or multimedia content** while on the move.

It's called a **"terminal"** because *it acts as the endpoint in a communication system the point where human interaction meets digital infrastructure.*

From a systems view, a mobile terminal is made up of:

- ❖ **Hardware:** CPU, memory, communication modules (**cellular, Wi-Fi, Bluetooth**), sensors, battery.
- ❖ **Operating System & APIs:** Provide the environment for running applications and interacting with the hardware.
- ❖ **Interface:** Touchscreen or keypad for user input, display for output.
- ❖ **Connectivity:** The ability to communicate over networks like **3G/4G/5G, NFC, or satellite.**

Mobile terminals are an integral part of modern society, ranging from common consumer devices to specialized equipment used in various industries. Here are some examples:

1.1.1. Consumer-Grade Mobile Terminals

These are the devices most people use in their daily lives.

- i. **Smartphones:** Being the most common example of a mobile terminal. Devices like the **Samsung Galaxy Ultra, Apple iPhone, or the Google Pixel** all serve as **personal computers, communication devices, cameras, and entertainment hubs all in one.**



- ii. **Tablets:** Devices such as the **Apple iPad**, **Samsung Galaxy Tab** or the **Google Pixel Tablet** offer a larger screen and more robust functionality than a smartphone, making them suitable for content consumption, creative work, and some professional tasks.



- iii. **Laptops:** A laptop also called a **notebook computer** is a portable personal computer designed for mobility and independent use. It integrates all the essential components of a desktop computer into a single compact unit. Laptops are powered by rechargeable batteries and can operate without being plugged in, making them ideal for mobile work, study, and entertainment. Notable examples include the **MacBook** and **HP computers**.



Even though some definitions of mobile devices emphasize “**handheld**” size, the broader understanding especially in mobile computing and network architecture includes laptops as mobile terminals. These devices are not just mobile they’re communication hubs, development platforms, and learning tools.

- iv. **Smart glasses:** Smart glasses are wearable computer-enabled eyewear that blend traditional optical lenses with digital technology to deliver real-time information, interaction, and multimedia experiences directly into the user's field of vision or auditory space. Smart glasses are designed to augment human perception by overlaying digital content onto the real world or providing hands-free access to digital services.

For instance, the **Ray-Ban Meta** can be considered a mobile terminal, albeit in a specialized and minimalist form. With the **First Generation (Ray-Ban Stories)** announced **September 2020**, but later on released **September 9, 2021** and the **Second Generation (Ray-Ban Meta)** announced **September 27, 2023**, and later on released on **October 17, 2023**. As we already know, a mobile terminal typically refers to a portable device that can **connect to networks, process data, and interact with users**. The Ray-Ban Meta glasses meet these criteria through:

- ❖ **Wireless connectivity:** Wi-Fi 6 and Bluetooth 5.3 support
- ❖ **Onboard processing:** Qualcomm Snapdragon AR1 Gen1 chip
- ❖ **Input/output interfaces:** Touchpad, voice commands, camera, microphone, and speakers
- ❖ **Integration with cloud services:** Meta AI and livestreaming to Facebook/Instagram

They're not general-purpose computing devices like smartphones, but they are **networked, wearable, and interactive**, making them a lightweight mobile terminal optimized for **media capture, communication, and AI assistance**.



In order to support the **VR glasses**, Meta created the **EMG (Electromyography) Neural Band**. It lets you control the interface with small hand movements, the experience feels seamless, without needing to touch a screen or speak.

- v. **Smartwatches and Wearables:** These are terminals designed to be worn on the body. Apple Watch, Fitbit, and other smartwatches provide on-the-go access to notifications, health monitoring, and even contactless payment.



1.1.2. Enterprise and Specialized Mobile Terminals

These devices are purpose-built for specific industries and tasks, often with a focus on durability, data collection, and integration with business systems.

- vi. **Mobile Point-of-Sale (mPOS) Systems:** These are mobile devices, often tablets or dedicated handhelds, that allow businesses to process transactions anywhere. Examples include the terminals used by **food trucks**, **retail employees on the sales floor**, or **delivery drivers to accept payments**.



- vii. **Rugged Handheld Computers:** Used in demanding environments like **warehouses**, **logistics**, and **manufacturing**, these devices (like those from **Zebra Technologies**) are built to withstand drops, extreme temperatures, and dust. They typically feature integrated barcode scanners and are used for inventory management and asset tracking.



- viii. **Vehicle-Mounted Terminals:** These are rugged computers mounted inside vehicles for **fleet management, navigation, and communication** with a central dispatcher. They are commonly found in **police cars, ambulances, and commercial delivery trucks**.



- ix. **Mobile Data Terminals (MDTs):** A more general term for devices used by field personnel. For example, a utility worker might use an MDT to **read meters**, and a **field technician might use one to access work orders and report on-site issues**.



As clearly portrayed above, mobile terminals are a category of portable electronic devices, which encompasses a wide range of devices from **smartphones and tablets to specialized devices like mobile point-of-sale (POS) systems and rugged data collectors**. Their features can be broadly categorized into **hardware, software, and connectivity** as explained below.

1.2. FEATURES OF MOBILE TERMINALS

i. Hardware Features

- ❖ **Display and Input:** Mobile terminals feature displays of varying sizes and technologies, with touchscreens being the most common input method. **Specialized terminals may also include physical keyboards or buttons for data entry in specific environments.**
- ❖ **Processor and Memory:** They are equipped with a **Central Processing Unit (CPU)** for processing tasks and a memory (**RAM**) for running applications. The amount of processing power and memory varies significantly depending on the device's intended purpose.

Modern mobile terminals are packed with sensors, including:

- ❖ **GPS:** For location tracking, navigation, and geolocating data.
- ❖ **Accelerometers and Gyroscopes:** To detect motion, orientation, and for use in gaming or augmented reality applications.
- ❖ **Cameras:** For capturing images and videos, which can be used for tasks like barcode scanning or document capture.
- ❖ **Scanners:** Specialized terminals often have integrated **1D and 2D barcode scanners, RFID readers (Radio-Frequency Identification)**, and/or magnetic stripe readers for business applications.
- ❖ **Durability:** Many industrial and enterprise-grade mobile terminals are designed to be rugged. They are built to withstand harsh conditions, including drops, extreme temperatures, dust, and water exposure, and are often rated with an **IP (Ingress Protection) code**.
- ❖ **Battery:** They are powered by batteries, typically **lithium-ion**, that are designed to provide power for a full workday or even longer, depending on the device and its usage.

ii. Software Features

- ❖ **Operating Systems (OS):** The most common operating systems for consumer-grade mobile terminals are Android and iOS. Enterprise-level terminals often use a customized version of an OS (like Android) tailored for specific business needs.
- ❖ **User Interface (UI):** A key feature of mobile terminals is their user-friendly, intuitive interfaces that allow for quick and easy interaction, which is crucial for both consumer and professional applications.
- ❖ **Security:** As mobile terminals handle sensitive data, they are designed with security in mind. Features include end-to-end encryption for data transmission, biometric authentication (fingerprint, facial recognition), and compliance with industry standards like PCI DSS for payment processing.

- ❖ **Application Support:** The ability to run a wide range of applications, either from a public app store or custom-developed for a specific enterprise, is a fundamental feature. This allows the devices to be adapted to a variety of tasks, from inventory management to mobile payment processing.

iii. Connectivity Features

- ❖ **Cellular Connectivity:** Most mobile terminals are capable of connecting to cellular networks (e.g., 4G LTE, 5G), allowing them to function in remote locations without Wi-Fi.
- ❖ **Wireless Communication:** They support Wi-Fi and Bluetooth for local network and peripheral connectivity (e.g., headsets, external printers, or scanners).
- ❖ **Near-Field Communication (NFC):** NFC is used for short-range communication, enabling features like contactless payments and data exchange with other NFC-enabled devices.
- ❖ **Ports and Peripherals:** Mobile terminals can connect to a variety of peripherals through physical ports (e.g., USB-C) or wirelessly. This allows them to interface with cash drawers, receipt printers, and other equipment to create a complete mobile workstation.

Thus, so far mobile terminals have become essential tools in both personal and professional spheres, but their unique nature presents a mix of significant advantages and notable disadvantages which include:

1.2.1. Advantages of Mobile Terminals

- ❖ **Unparalleled Portability and Mobility:** The primary advantage of mobile terminals is their compact size and wireless nature. This allows users to access information, communicate, and perform tasks from virtually anywhere, without being tethered to a desk. This mobility is crucial for field workers, sales teams, and medical professionals who need to be productive on the move.
- ❖ **Real-Time Data Access and Collection:** With constant internet connectivity (via Wi-Fi or cellular networks), mobile terminals provide immediate access to real-time data. This capability is vital for applications like mobile point-of-sale (mPOS) systems, where inventory and sales data must be updated instantly, or for delivery services that require real-time tracking and communication.
- ❖ **Versatility and Multipurpose Functionality:** Modern mobile terminals are incredibly versatile. A single device can function as a communication tool (phone, email), a computer, a camera, a GPS navigator, a barcode scanner, and a payment processor. This consolidation of tools into one device simplifies workflows and reduces the need for multiple pieces of equipment.
- ❖ **Intuitive User Interfaces:** The touch-based user interfaces of most mobile terminals are highly intuitive and easy to learn. This reduces the training time required for new users and makes them accessible to a broad demographic, from young children to the elderly. In a business setting, this translates to faster employee adoption and reduced training costs.
- ❖ **Enhanced Productivity and Efficiency:** By providing tools for communication, data access, and task management in a single portable device, mobile terminals empower

users to be more efficient. They enable quicker decision-making, faster data entry, and streamlined processes, ultimately boosting overall productivity.

1.2.2. Disadvantages of Mobile Terminals

- ❖ **Limited Screen Size and Input Constraints:** Compared to a desktop computer, the small screen size of most mobile terminals can make it challenging to view complex data or documents. The lack of a physical keyboard and mouse can also slow down tasks that involve extensive data entry, writing, or precision-based work.
- ❖ **Security Risks and Vulnerability:** Due to their portability, mobile terminals are more susceptible to being lost or stolen. This creates a significant security risk, as a stolen device can contain sensitive personal or company data. They are also targets for malware and phishing attacks, which can compromise security if not properly managed.
- ❖ **Battery Life Limitations:** Mobile terminals rely on rechargeable batteries, and their usage is directly tied to battery life. Heavy use of demanding applications, constant connectivity, or bright screen settings can quickly drain the battery, leading to interruptions in work and a need for frequent recharging.
- ❖ **Hardware and Performance Constraints:** While modern mobile processors are powerful, they generally cannot match the processing power or memory capacity of a desktop or laptop computer. This can limit their use for high-performance computing tasks, complex data analysis, or professional-grade creative work.
- ❖ **Distractions and Reduced Focus:** The constant stream of notifications, easy access to social media, and a wide array of entertainment apps on mobile terminals can be a significant source of distraction. In a professional environment, this can lead to reduced focus and a decline in productivity.

Conclusion

In Cameroon, mobile terminals are transforming access to **education, healthcare, and financial services**. Initiatives like **smart classrooms, digital health records, and mobile learning apps** are powered by these devices. Understanding mobile terminals helps **educators, as well as developers** to design solutions that meet local needs. Therefore, we can agree on the fact that mobile terminals are the gateway to the digital world. For a beginner, mastering the basics of *what they are, how they work, and how to use them safely* opens the door to more advanced topics like **mobile app development, network optimization, and device troubleshooting**. In Cameroon, understanding mobile terminals is not just a tech skill, it's a tool for **education, business, and social impact**.

MODULE 02

INTRODUCTION TO MOBILE OPERATING SYSTEMS

2.1. INTRODUCTION

An Operating System (OS) is the foundational software that manages computer hardware and provides services for computer programs. Think of it as the bridge between you and your machine it makes everything work together smoothly. As core functions, here's what an OS does behind the scenes:

2.1.1. Core Functions of an Operating System

- ❖ **File Management:** Organizes, stores, retrieves, and secures data on storage devices.
- ❖ **Process Management:** Handles running applications, multitasking, and allocating CPU time.
- ❖ **Memory Management:** Allocates RAM to programs and ensures efficient memory usage.
- ❖ **Device Management:** Controls input/output devices like keyboards, printers, and monitors.
- ❖ **Security & Access Control:** Protects system resources and user data from unauthorized access.
- ❖ **User Interface:** Provides a way for users to interact with the system either through a Graphical User Interface (GUI) or Command-Line Interface (CLI).

2.1.2. Types of Operating Systems

TYPE	DESCRIPTION
Desktop OS	Used in personal computers (e.g., Windows, macOS, Linux)
Mobile OS	Designed for smartphones and tablets (e.g., Android, iOS)
Server OS	Optimized for managing network resources (e.g., Windows Server, Ubuntu Server)
Embedded OS	Runs on specialized devices like smart TVs, ATMs, or IoT gadgets
Real-Time OS (RTOS)	Used in systems requiring immediate response (e.g., robotics, medical devices)

From the table above it can be understood that a mobile operating system (OS) is a system software that allows a mobile device, such as a smartphone or tablet, to run applications and perform its core functions. It acts as the bridge between the device's hardware and the user, managing everything from the user interface and memory to security and connectivity. Modern mobile operating systems are optimized for features like **touchscreens, cellular connectivity, and a variety of sensors (e.g., GPS, accelerometers)**.

Here is a detailed explanation of the most prominent mobile operating systems, as well as some other notable examples.

2.1.2.1. Android OS

Android is the world's most popular mobile operating system. Developed by **Google** and based on a modified **Linux kernel**, Android is an **open-source platform**.

Its open-source nature means that device manufacturers can customize the software for their specific hardware, leading to a vast and diverse ecosystem of **phones, tablets, smartwatches, and even smart TVs**. Android devices are available from a wide range of brands, including **Samsung, Google, and OnePlus**, spanning a broad spectrum of price points.

NB: Market Share: As of late 2024, Android holds a dominant global market share, controlling roughly 70% of the mobile OS market worldwide.

2.1.2.2. Key Features of the Android OS

- ❖ **Customizability:** Users can personalize their home screens with widgets, change app icons, and install third-party launchers, giving them a high degree of control over the user experience.
- ❖ **Integration with Google Services:** It offers seamless integration with Google's suite of services, including **Gmail, Google Maps, Google Drive, and Google Gemini**.
- ❖ **Vast App Ecosystem:** The Google Play Store offers millions of applications, and because of its open nature, users can also download apps from other sources.

2.1.2.3. Advantages and Disadvantages of the Android OS

Advantages

Surely its greatest advantages we can talk about is its open nature which allows/permits for more hardware choices and greater customization. Android is also the global market leader, making it a powerful platform for developers.

- ❖ **Open-Source and High Customization:** Android's open-source nature is its most significant advantage. Device manufacturers can customize the OS to fit their hardware, leading to a vast array of unique phones, tablets, and other devices. Users also benefit from this, with the ability to deeply customize their user interface through widgets, launchers, and icon packs. This flexibility allows for a highly personalized mobile experience.
- ❖ **Wide Variety of Hardware and Price Points:** Because Android is available to any hardware manufacturer, there is an incredible diversity of devices on the market. This creates strong competition, which drives down prices and offers consumers a wide range of choices, from premium flagship phones to affordable budget devices. This makes Android accessible to a much broader global audience.
- ❖ **Flexibility and User Freedom:** Android offers more freedom to its users than a closed system. Users can easily sideload applications from sources outside the Google Play Store, customize default apps, and even replace the entire operating system with a custom ROM. This level of control is appealing to power users and developers.

- ❖ **Integration with Google's Ecosystem:** For users who rely on Google services, Android provides seamless integration with products like **Gmail, Google Maps, Google Drive, and Google Assistant**. This creates a cohesive experience for managing personal and professional life across different devices.
- ❖ **Robust App Ecosystem:** The Google Play Store is home to millions of apps. Due to the OS's dominant market share, most developers prioritize creating and optimizing their apps for Android, ensuring a rich and diverse selection for users.

Disadvantages

But sadly, its greatest advantage can equally lead to a massive disadvantage whereby its open nature also leads to "**fragmentation**," where many devices run on different versions of the OS, which can create security vulnerabilities and inconsistencies in the user experience.

- ❖ **Fragmentation:** This is Android's most notorious drawback. Because the OS is used by many different manufacturers on countless devices, there are multiple versions of Android in active use at any given time. This makes it challenging for developers to ensure their apps work perfectly on all devices and can lead to security vulnerabilities, as older versions may not receive timely security patches.
- ❖ **Slower and Inconsistent Updates:** While Google releases new versions of Android annually, device manufacturers and mobile carriers are responsible for pushing these updates to their devices. This often results in significant delays, with some devices waiting months or even years for a new OS version, and many older devices never receiving an update at all. This leaves users with outdated features and security gaps.
- ❖ **Security Risks:** The open nature of Android makes it a more frequent target for malware. While Google Play Protect helps, the ability to download apps from outside the Play Store and the fragmentation of the OS mean that some users may be more exposed to malicious software. Security reports consistently show that Android has a higher volume of malware compared to iOS.
- ❖ **Potential for Bloatware:** Device manufacturers and carriers often pre-install their own apps (bloatware) on Android devices. These apps can consume storage space and system resources, and in many cases, they cannot be uninstalled by the user, leading to a cluttered and sometimes sluggish experience.

2.1.2.4. History and Evolution of the Android OS

The Android OS was originally developed by **Android Inc.**, a company founded in **2003** by **Andy Rubin, Rich Miner, Nick Sears, and Chris White**. Interestingly, their initial goal was to create an operating system for **digital cameras**, but they soon pivoted to focus on **smartphones** as the market shifted.

In 2005, **Google** acquired Android Inc., and the development of the OS accelerated under its leadership. Google chose to base Android on the **Linux kernel**, making it open-source and freely available to device manufacturers. This strategic move helped Android become the most widely used mobile operating system globally.

The evolution of the Android operating system is a testament to Google's commitment to continuous innovation. From its humble beginnings in **2008**, Android has transformed from a basic mobile platform into a sophisticated, feature-rich OS that dominates the global market.

Here is a chronological look at the evolution of Android OS.

i. The Early Years (2008 - 2011): The Foundations

- ❖ **Android 1.0 (2008):** This was the first commercial release, launching on the HTC Dream (T-Mobile G1). It laid the groundwork with core features like a **pull-down notifications bar**, integration with Google services (Gmail, Maps, Calendar), and the Android Market (the precursor to the Google Play Store).



- ❖ **Android 1.5 "Cupcake" (2009):** This version introduced the first **on-screen keyboard**, **widgets**, and the **ability to record videos**, making it a major step forward in usability.



- ❖ **Android 1.6 "Donut" (2009):** Key additions included support for **multiple screen resolutions**, which was crucial for device diversity, and an **improved camera and gallery**. It also added support for **CDMA networks**, significantly expanding Android's market reach.



Donut

Android 1.6

F.

NB: GSM and CDMA are two different technologies used in cellular networks. GSM (Global System for Mobile Communications) is a standard that uses a SIM card to identify subscribers and enable roaming. CDMA (Code Division Multiple Access) is another standard where phones are linked to the network via a phone number and authentication is handled by the carrier.

- ❖ **Android 2.0/2.1 "Eclair" (2009-2010):** A major visual and functional overhaul. Eclair brought a **revamped UI, support for multiple Google accounts, and live wallpapers**. It also introduced **HTML5** support and **Bluetooth 2.1**, improving web browsing and connectivity.



Eclair

Android 2.0/2.1

- ❖ **Android 2.2 "Froyo" (2010):** This version focused heavily on performance improvements, with a new JIT (Just-In-Time) compiler that significantly sped up app execution. It also added features like **Wi-Fi hotspot functionality and push notifications**.



Froyo
Android 2.2/2.2.3

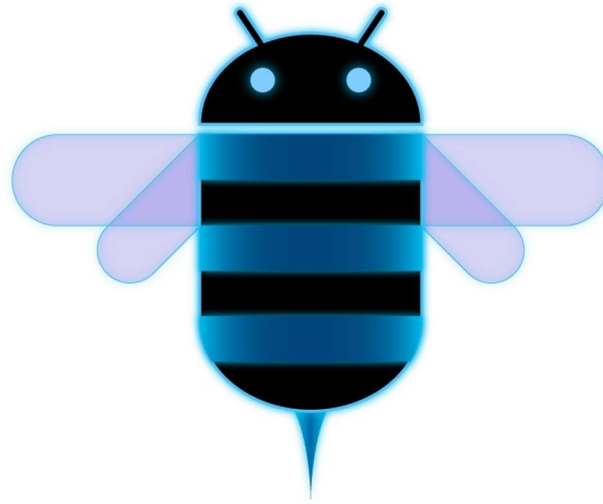
- ❖ **Android 2.3 "Gingerbread" (2010):** A massive update that brought a new, more refined UI. It optimized power management, introduced support for NFC, and added a native copy/paste function. This version solidified Android's place as a serious competitor to other mobile OSs.

Android 2.3



ii. **The Tablet Era & Design Overhauls (2011 - 2013)**

- ❖ **Android 3.0 "Honeycomb" (2011):** This was a tablet-only version designed to take advantage of larger screens. It introduced a new "Holographic" UI, a redesigned status bar, and a multitasking button.



- ❖ **Android 4.0 "Ice Cream Sandwich" (2011):** This marked a pivotal moment by unifying the smartphone and tablet OS into a single platform. It introduced the modern, card-like **"Recent Apps"** menu, **Face Unlock**, and **on-screen navigation buttons**, replacing the physical keys of old.



- ❖ **Android 4.1-4.3 "Jelly Bean" (2012-2013):** This series focused on performance with "Project Butter," which made the UI smoother and more responsive by using a triple-buffering system. It also introduced **Google Now**, a **predictive digital assistant**, and **expandable notifications**.



ANDROID 4.1
JELLY BEAN

- ❖ **Android 4.4 "KitKat" (2013):** A major optimization that allowed Android to run on a wider range of devices with as little as **512MB** of RAM, helping to reduce OS fragmentation. It also brought a more **transparent UI** and the **"Ok Google" voice command on the home screen**.



ANDROID
4.4 KitKat

iii. **The Modern Era (2014 - Present)**

- ❖ **Android 5.0 "Lollipop" (2014):** This was a complete visual redesign, introducing "Material Design," a bold, colourful, and fluid design language. It also switched to the ART (Android Runtime) by default, leading to further performance gains, and introduced a new notification system with lock screen alerts.



Android 5.0, Lollipop

- ❖ **Android 6.0 "Marshmallow" (2015):** Focused on polishing the user experience. Key features included a **new app drawer, native fingerprint sensor support, and Doze Mode**, a battery-saving feature that put the device into a deeper sleep state when inactive.



- ❖ **Android 7.0 "Nougat" (2016):** This version brought a number of key usability features, most notably **Split-Screen Multitasking, direct replies from notifications, and improved background performance.**



- ❖ **Android 8.0 "Oreo" (2017):** Focused on refining the platform. It introduced **"Notification Channels" for granular control over notifications, a Picture-in-Picture mode, and improved boot times.**



- ❖ **Android 9 "Pie" (2018):** This OS embraced gesture navigation, replacing the traditional three-button navigation bar. It also introduced **Digital Wellbeing tools to help users manage screen time and "Adaptive Battery," which used machine learning to prioritize apps for battery life.**



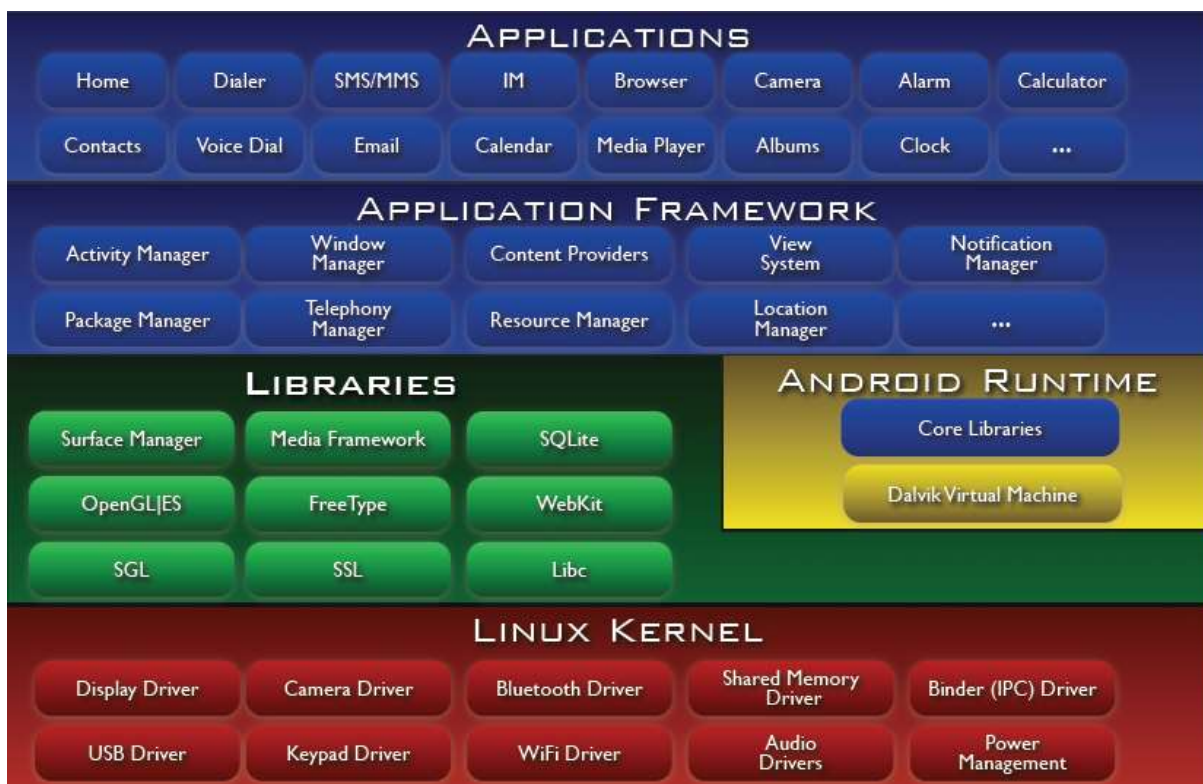
- ❖ **Android 10 (2019):** Google switched to a numerical naming convention. This version brought a **system-wide Dark Theme**, a more robust gesture navigation system, and significant improvements to **privacy controls**, giving users more control over location and app permissions.
- ❖ **Android 11 (Red Velvet Cake) (2020):** Key features included **"Conversations"** in the notification shade, a single spot for chat notifications from different apps, a **"Bubbles"** chat feature, and enhanced privacy tools with one-time permissions.
- ❖ **Android 12 (Snow Cone) (2021):** A major visual update with Material You, a design language that automatically changes the colour of the UI based on your wallpaper. It also added a new privacy dashboard and improved security features.
- ❖ **Android 13 (Tiramisu) (2022):** This version continued to build on Material You, offering more colour options. It improved media controls, added per-app language settings, and introduced stricter privacy controls for media access.
- ❖ **Android 14 (Upside Down Cake) (2023):** This update introduced new accessibility features like font scaling up to 200%, improved battery life, and enhanced the platform's ability to run on large-screen devices like foldables and tablets.
- ❖ **Android 15 (Vanilla Ice Cream) (2024):** The latest stable release. Android 15 brings features like Private Space, a secure, separate container for sensitive apps and data; satellite connectivity for messaging; and improved support for third-party camera apps with better low-light performance.
- ❖ **Android 16 (Baklava):** As of now, Android 16 is in development. Developer previews have shown a focus on improved performance profiling for developers, a revamped system for notifications with "live updates," and further enhancements to the "predictive back" gesture. Its full capabilities will be revealed upon its official release.

As of **January 2026**, Android 15 is the current stable version found on most flagship devices. However, looking ahead at the release cycle:

- ❖ **Android 15 (Vanilla Ice Cream):** The current mainstream version. It introduced enhanced features like "Private Space" (hiding apps), partial screen sharing, and better support for satellite connectivity.
- ❖ **Android 16 (Baklava):** Google has recently shifted its development timeline. Android 16 is expected to be released earlier in the year (likely mid-2025) compared to previous versions, meaning by early 2026, many devices may already be receiving this update.

2.2. THE ANDROID ARCHITECTURE

The Android architecture is a layered stack of software components, designed to provide a cohesive and flexible environment for mobile applications. It's built to enable a wide variety of devices to run the same operating system while allowing developers to create applications without needing to understand the low-level hardware specifics.



As shown on the diagram above, the architecture is typically broken down into five main layers, which are explained below

i. The Linux Kernel

At the very bottom of the Android software stack is the **Linux Kernel**. This is the foundation of the Android OS, acting as the interface between the device's hardware and the rest of the software. It handles fundamental system tasks that are critical for the device to function.

Key Functions:

- ❖ **Hardware Drivers:** Contains all the drivers for the device's hardware components, such as the **display, camera, Wi-Fi, audio, and Bluetooth**.

- ❖ **Memory Management:** Manages the device's memory, ensuring that apps have the resources they need to run without crashing the system.
- ❖ **Process Management:** Controls how processes are scheduled on the CPU, allowing multiple applications to run at the same time (**multitasking**).
- ❖ **Power Management:** Manages power consumption to extend battery life.

ii. Hardware Abstraction Layer (HAL) & Native Libraries

Directly above the Linux Kernel are the **Hardware Abstraction Layer (HAL)** and **Native Libraries**. This layer is crucial for Android's ability to run on a wide variety of hardware.

- a. **Hardware Abstraction Layer (HAL):** The HAL provides a standard set of interfaces that the Android framework uses to communicate with device-specific hardware. It acts as a bridge, allowing the Android OS to work with different hardware without needing to know the complex details of how that hardware operates. For example, a camera HAL module provides a uniform interface for the Android system to use the camera, regardless of whether the physical camera is from Samsung, Sony, or any other manufacturer.
- b. **Native Libraries:** These are libraries written in **C/C++** that are used by various Android components. They are essential for performance-critical tasks. Examples include:
 - ❖ **SurfaceFlinger:** Manages graphics and is responsible for composing all the application and system UI on the screen.
 - ❖ **OpenGL ES:** Provides APIs for 2D and 3D graphics rendering.
 - ❖ **SQLite:** A lightweight database engine used for local data storage by many applications.

iii. Android Runtime (ART)

The Android Runtime (ART) is the environment in which Android applications run. Before Android 5.0, this was the Dalvik Virtual Machine (DVM). ART replaced Dalvik to improve app performance and battery life.

Key Functions:

- ❖ **Ahead-of-Time (AOT) Compilation:** Unlike Dalvik, which compiled code just-in-time, ART compiles an application's code into native machine code when it's installed. This allows apps to launch and run much faster.
- ❖ **Just-in-Time (JIT) Compilation:** Starting with Android 7.0, ART also uses JIT compilation to dynamically optimize frequently used code during runtime, which helps save storage space and further improves performance.
- ❖ **Garbage Collection:** ART has an efficient garbage collector that reclaims memory from apps that are no longer in use, ensuring the system remains responsive.
- ❖ **Core Libraries:** This component contains the core Java APIs and libraries that provide developers with the tools to build applications.

iv. Application Framework

The Application Framework is a key layer for developers. It's a collection of high-level services and APIs that developers can use to build Android applications. This is where the core Android components like **Activities** and **Services** are defined.

Key Services and Managers:

- ❖ **Activity Manager:** Manages the lifecycle of applications, including starting, stopping, and switching between activities.
- ❖ **Window Manager:** Manages the graphical windows and views on the screen.
- ❖ **Content Providers:** A standard way for applications to share data with other applications securely.
- ❖ **Notification Manager:** Handles the creation and management of notifications that appear in the status bar.
- ❖ **Resource Manager:** Provides access to non-code resources like layouts, images, and strings.

v. Applications

This is the top layer of the Android architecture. It consists of all the applications that users install and interact with. This includes both the **pre-installed System Apps (like the phone dialler, contacts, and camera)** and all the **third-party apps downloaded from the Google Play Store**.

- ❖ **Interaction:** Applications are built using the components and services provided by the Application Framework. They are developed using languages like **Kotlin and Java** and are packaged as **APK files**. The **Android Runtime (ART)** is the environment where these applications execute.

Question 1: Due to its open nature what are some mobile security threats usually associated with the Android OS and the devices on which they are deployed?

Mobile security threats can be categorized into two primary types: **physical threats and software-based threats**. In order to protect your device and personal data, you must understand each category and its corresponding countermeasures.

Physical Threats

Physical threats involve unauthorized access to a mobile device through direct, physical contact. The most common threats are loss and theft.

Threats

- ❖ **Device Theft:** When a mobile device is stolen, the attacker can attempt to bypass the lock screen to access personal information, including contacts, emails, passwords, and banking details.
- ❖ **Loss of Device:** An unsecure, lost device poses the same risks as a stolen one. Anyone who finds it can potentially gain access to the data on it, or they can simply wipe the device and resell it.

Ways to Combat Them

- ❖ **Use Strong Authentication:** Always use a **strong passcode, PIN, or passphrase**. For added security and convenience, enable **biometric authentication like fingerprint or facial recognition**.

- ❖ **Enable Remote Lock and Wipe:** Both Android and iOS have built-in features (**Find My Device** and **Find My iPhone**, respectively) that allow you to **remotely locate, lock, or erase all data from a lost or stolen device**. It's crucial to have this feature set up before a device is lost.
- ❖ **Regular Data Backups:** Regularly back up your data to a secure cloud service (e.g., **Google Drive, iCloud**) or an **external hard drive**. This ensures that even if your device is compromised, you can restore your data to a new device.

Software-Based Threats

Software-based threats exploit vulnerabilities in the operating system, applications, or network connections.

a. Malware

Malware is a broad term for **malicious software** designed to disrupt, damage, or gain unauthorized access to a device.

Threats

- ❖ **Viruses and Trojans:** These can hide in seemingly legitimate apps and steal personal information, monitor your activity, or use your device to send spam messages.
- ❖ **Ransomware:** This type of malware encrypts your files and holds them hostage, demanding a ransom (**usually in cryptocurrency**) for their release.
- ❖ **Spyware:** Spyware operates in the background, secretly collecting data such as your browsing history, contacts, messages, and passwords.

Ways to Combat Them

- ❖ **Install Apps from Trusted Sources:** Only download apps from official app stores like the Google Play Store or the Apple App Store, as these platforms have security measures to vet applications. Avoid third-party app stores or websites.
- ❖ **Review App Permissions:** Be cautious of apps that request excessive permissions, such as a simple calculator app that wants access to your contacts or location.
- ❖ **Install Antivirus/Anti-Malware Software:** Consider installing a reputable mobile security app to scan for and block malicious software in real-time.
- ❖ **Keep Your OS and Apps Updated:** Software updates often include critical security patches that fix vulnerabilities. Enable automatic updates for both your OS and all your applications.

b. Phishing

Phishing is a form of **social engineering** where attackers trick you into revealing sensitive information.

Threats

- ❖ **SMS Phishing (Smishing):** Scammers send text messages with malicious links, often disguised as alerts from a bank or a popular service, to trick you into entering login credentials.

- ❖ **Email and In-App Phishing:** Phishing emails or pop-ups within apps can imitate a trusted company and pressure you into providing personal data.

Ways to Combat Them

- ❖ **Be Sceptical of Suspicious Links:** Never click on links in unsolicited emails or text messages, even if they appear to be from a known contact.
- ❖ **Verify the Source:** If a message from a bank or service seems urgent, go directly to the company's official website or app instead of clicking the link. Legitimate companies rarely ask for sensitive information via email or text.
- ❖ **Use Multi-Factor Authentication (MFA):** Enable MFA on all your important accounts. This requires a second form of verification (**like a code from an authenticator app such Google Authenticator or Microsoft Authenticator**) in addition to your password, making it much harder for an attacker to gain access even if they steal your credentials.

c. Network-Based Threats

These threats exploit vulnerabilities in network connections, especially on public networks.

Threats

- ❖ **Man-in-the-Middle (MitM) Attacks:** On an unsecured public Wi-Fi network, an attacker can intercept the data traveling between your device and the internet, gaining access to your personal information.
- ❖ **Malicious Hotspots:** An attacker can set up a fake Wi-Fi network with a common name (e.g., "Free Airport WI-FI") to trick users into connecting and revealing their data.

Ways to Combat Them

- ❖ **Avoid Public Wi-Fi for Sensitive Tasks:** Do not access banking, shopping, or other sensitive accounts on an unsecured public Wi-Fi network.
- ❖ **Use a VPN:** A Virtual Private Network (VPN) encrypts all data leaving your device, creating a secure tunnel. This makes it impossible for attackers to intercept and read your data, even on an unsecure public network.
- ❖ **Turn Off Wi-Fi and Bluetooth:** When not in use, turn off your device's Wi-Fi and Bluetooth to prevent it from automatically connecting to malicious hotspots or being exploited through an open connection.

2.3. THE iOS ENVIRONMENT

iOS (formerly iPhone OS) is a mobile operating system created and developed by **Apple Inc.** exclusively for its line of iPhones and **iPads (until iPadOS split off), and iPod Touch.** Introduced in **2007** with the **first iPhone** by then CEO late **Steve Jobs**, it is a proprietary, closed-source system that has become one of the world's most widely used mobile operating systems.

NB: The first iPhone, officially known as the iPhone (1st generation), was released by Apple on June 29th, 2007. It marked a revolutionary shift in mobile technology and laid the foundation for the modern smartphone era

Steve Jobs famously introduced it as:

“An iPod, a phone, and an internet communicator. Are you getting it? These are not three separate devices. This is one device.” Below is an image of the very first iPhone:



2.3.1. Key Features of the First iPhone

FEATURE	DETAILS
Display	3.5-inch multi-touch LCD (480×320 resolution)
Storage Options	4 GB, 8 GB, later 16 GB
Camera	2.0 MP rear camera (no front camera)
Operating System	iPhone OS 1.0 (later updated to iPhone OS 3.1.3)
Processor	Samsung S5L8900 (ARM-based)
RAM	128 MB eDRAM
Connectivity	Quad-band GSM, EDGE, Wi-Fi (802.11b/g), Bluetooth 2.0
Battery	1400 mAh lithium-ion
Form Factor	Slate design, 115 × 61 × 11.6 mm, 135 g weight

Question 2: As being the very first iPhone released what were the innovations introduced as compared to the already existing brands back then?

- ❖ **Multi-touch interface:** No stylus or physical keyboard just finger gestures.
- ❖ **Visual voicemail:** A new way to manage voice messages.
- ❖ **Safari browser:** Full web browsing on a mobile device.
- ❖ **iPod integration:** Music and video playback built-in.
- ❖ **Unified device:** Combined phone, iPod, and internet communicator.

From there on **over 6 million units** were sold before being discontinued in **July 2008**. It equally set the stage for the **App Store**, which launched with the **iPhone 3G**.

As a lasting legacy it pioneered the design language and user experience that influenced all future smartphones. Below is video that explains the history and evolution of the iPhone.

[Evolution of the iPhone \(NEW\) \(2007-2025\)](#)

Before the first iPhone's introduction in **2007**, the smartphone market was a collection of devices that combined phone capabilities with features from **Personal Digital Assistants (PDAs)**. These phones were primarily targeted at business users and early adopters, and their design and functionality were very different from the seamless, touchscreen-centric experience the iPhone introduced.

Below, are some of the dominant smartphone models and operating systems of that era:

BlackBerry OS

BlackBerry was the undisputed leader in the smartphone market, especially in the corporate world. Their devices were famous for their focus on secure, push-email services, which were



- ❖ **Key Features:** BlackBerry phones were defined by their physical **QWERTY keyboard**, which allowed for rapid typing. Navigation was handled by a **trackball or trackpad**, and the user interface was designed around **quick access to email, calendar, and contacts**.
- ❖ **Examples:** The **BlackBerry 8800** and **BlackBerry Pearl series** (portrayed above) were top models in 2007. While they offered email and web browsing, their small, non-touch screens and reliance on physical input made them a stark contrast to the iPhone's multi-touch screen.

Symbian OS

Nokia was the dominant mobile phone manufacturer and its smartphones primarily ran on **Symbian OS**. The Symbian platform was a powerful system that prioritized a wide range of features.



- ❖ **Key Features:** Symbian phones were known for their **excellent cameras, GPS capabilities, and robust multimedia support**. However, their user interface was often complex and not optimized for touch, with users navigating through a series of menus with a keypad.
- ❖ **Examples:** The **Nokia N95** was a high-end multimedia powerhouse with a 5-megapixel camera and a **unique dual-sliding design to reveal media controls**. It was a feature-rich device, but its user experience was far from the intuitive nature of the iPhone.

Windows Mobile

Microsoft's Windows Mobile platform attempted to bring the desktop experience to a mobile device. It was popular on a variety of devices from different manufacturers.



- ❖ **Key Features:** These devices typically featured a resistive touchscreen that required a stylus for navigation and data entry. The interface often resembled a scaled-down version of the Windows desktop, with a Start menu and tiny icons that were difficult to use with a finger.
- ❖ **Examples:** The **HTC Touch (as shown above)** was a notable Windows Mobile device. It was a finger-friendly attempt to skin Windows Mobile to make it more usable, but its underlying OS was still built for **stylus-based input**. The **Palm Treo series**, which ran on either Palm OS or Windows Mobile (**portrayed below**), was also a popular choice that combined a physical keyboard with a stylus-driven touchscreen.



Other Notable Devices

- ❖ **LG Prada (KE850):** Released just before the iPhone, the **LG Prada** was the first phone with a capacitive touchscreen (**As shown below**). While it offered a glimpse of a finger-friendly interface, it lacked a robust operating system and a wide range of apps, making it a "feature phone" rather than a true smartphone competitor.



The iPhone's innovation was not just in being a single device, but in its multi-touch screen and a simplified, unified software experience that made it easy for anyone to use. By combining a **music player, a phone, and an internet communicator into one device**, it redefined the smartphone and marked a clear departure from the feature-heavy but clunky devices that came before it.

2.3.2. Key Features of the iOS Environment

The iOS environment is a tightly integrated ecosystem known for its simplicity, security, and consistent user experience. It's built on the principle of a "**walled garden**," where **Apple maintains control over both the software and hardware to ensure optimal performance and security**.

i. User Interface and Design

iOS is renowned for its intuitive, fluid, and gesture-based user interface. The design is clean and minimalist, following principles of clarity, consistency, and simplicity. Users navigate using multi-touch gestures like tapping, swiping, and pinching. The core UI elements include:

- ❖ **Home Screen:** The primary hub for app icons and widgets.
- ❖ **Notification Centre:** A centralized location for all alerts, with an option to group notifications by app.
- ❖ **Control Centre:** A customizable panel for quick access to essential settings like Wi-Fi, Bluetooth, and screen brightness.
- ❖ **Face ID and Touch ID:** Biometric authentication systems for unlocking the device, authorizing payments, and logging into apps.

ii. Ecosystem and Integration

A key feature of the iOS environment is the seamless integration between Apple's devices and services. This synergy makes it easy to move between a **Mac, iPhone, iPad, and Apple Watch**.

- ❖ **iCloud:** A cloud service that automatically synchronizes data like your photos, contacts, calendars, and messages across all of your Apple devices.
- ❖ **Continuity:** This suite of features allows for seamless transitions between devices. For example, **Handoff** lets you start a task on one device and continue it on another, while **Universal Clipboard** allows you to copy content on one device and paste it on another.
- ❖ **Airdrop:** A wireless feature that enables fast and secure sharing of photos, videos, documents, and other files between nearby Apple devices.
- ❖ **The App Store:** The sole official marketplace for iOS applications. Apple's rigorous app review process ensures a high standard of security and quality, protecting users from malware and poorly developed apps.

iii. Security and Privacy

iOS is widely regarded for its security, largely due to its closed-source nature and robust security architecture.

- ❖ **Secure Enclave:** A dedicated, isolated processor that stores biometric data (**Face ID and Touch ID**) and cryptographic keys. This ensures that even if the main processor is compromised, your most sensitive data remains safe.
- ❖ **App Sandboxing:** Each application runs in its own isolated environment, preventing it from accessing data from other apps or the core operating system without explicit user permission.
- ❖ **Data Encryption:** All data stored on an iOS device is encrypted by default, and services like iMessage and FaceTime use end-to-end encryption to protect communications from interception.
- ❖ **App Tracking Transparency (ATT):** This feature requires apps to ask for user permission before tracking their activity across other apps and websites, giving users more control over their privacy.

iv. Apple Intelligence

Apple Intelligence is Apple's integrated AI system designed to enhance user experience across **iPhone, iPad, Mac, and Apple Vision Pro devices**. It was officially unveiled at **WWDC 2024** and began rolling out with **iOS 18, iPadOS 18, and macOS Sequoia**. As for

the iPhone the supported devices include the **iPhone 15 Pro, iPhone 15 Pro Max, iPhone 16 & 17 series (all models).**

These models feature **Apple's A17 Pro** or newer chips, which support on-device generative AI tasks like **summarization, Genmoji creation, and intelligent Siri enhancements.**

As for the iPad we have got the **M1 iPad Pro, M2 iPad Pro, M4 iPad Pro, M1 iPad Air, M2 iPad Air and iPad mini-7.** Apple Intelligence features are deeply integrated into iPadOS 18 and later, enabling tools like **Image Playground, Smart Replies, and document summarization.**

Meanwhile, the Mac includes the **MacBook Air: M1, M2, M3, MacBook Pro: M1, M2, M3** (including Pro and Max variants), **iMac: M1, M2, Mac mini: M1, M2 Max, M2 Ultra, Mac Studio: M1 Max, M1 Ultra, M2 Ultra, Mac Pro: Apple Silicon models.** These devices run **macOS Sequoia** and support Apple Intelligence features like **Writing Tools, Memory Maker in Photos, and intelligent automation across apps.**

Upon all these, Apple has released five generations of M-series chips, with multiple versions in each generation whereby we have got the;

Apple M1
2020-2022



November 2020



October 2021



October 2021



March 2022

Apple M2
2022-2023



June 2022



January 2023



January 2023



June 2023

Apple M3
2023-2025



October 2023



October 2023



October 2023



March 2025

Apple M4
2024



May 2024



October 2024



October 2024

Apple M5
2025



October 2025

- a. **M1:** The first generation, released in **November 2020**, that includes;
 - ❖ **M1**
 - ❖ **M1 Pro**
 - ❖ **M1 Max**
 - ❖ **M1 Ultra**

- b. **M2:** The second generation, released in **June 2022**, that includes;
 - ❖ **M2**
 - ❖ **M2 Pro**
 - ❖ **M2 Max**
 - ❖ **M2 Ultra**

- c. **M3:** The third generation, released in **October 2023**, that includes;
 - ❖ **M3**
 - ❖ **M3 Pro**
 - ❖ **M3 Max**
 - ❖ **M3 Ultra**

- d. **M4:** The fourth generation, released in **May 2024**, that includes;
 - ❖ **M4**
 - ❖ **M4 Pro**
 - ❖ **M4 Max**
 - ❖ **M4 Ultra**

- e. **M5:** The latest generation, announced in **October 2025**, includes;
 - ❖ **M5**
 - ❖ **M5 Pro**
 - ❖ **M5 Max**
 - ❖ **M5 Ultra**

That's a total of 15 different M-series chips, with more likely to come

But for the sake of clarity, **Apple Intelligence is Apple's branded suite of AI features introduced in the 2024/2025 software season.** It's not a different kind of AI, but rather Apple's philosophy and implementation of AI technologies across its ecosystem. Think of it as:

"AI for the rest of us" designed to be private, personal, and practical.

This video provides a deep dive and better understanding of what Apple Intelligence is and of what benefit it is to you.

[Hands on with Apple Intelligence | Apple](#)

Listed below are the core features of Apple Intelligence and what makes it stand out:

Writing Tools Everywhere

Apple Intelligence helps users write better by offering:

- ❖ Summarization of long texts (**emails, transcripts, group chats**)
- ❖ Tone adjustment and proofreading
- ❖ Smart replies that auto-detect questions and suggest responses These tools are embedded across apps like **Mail, Messages, Notes, and even third-party apps.**

Smart Notifications & Focus

It prioritizes urgent messages and summarizes notifications so users can focus on what matters. A new “Reduce Interruptions” mode filters out less relevant alerts.

Image Generation & Genmoji

Users can create personalized emojis (**called Genmoji**) and expressive images using prompts. The Image Playground app lets you generate visuals for use in Messages, Keynote, and more.

Revamped Siri

Siri now works across apps with contextual awareness. For example, you can ask Siri to edit a photo and insert it into a message all in one go. The assistant also responds based on what’s on your screen.

Privacy-Centric Design

Apple Intelligence uses a mix of on-device and cloud-based models, but all cloud processing happens on Apple’s own servers with custom silicon and end-to-end encryption. Apple emphasizes that even it cannot access your personal data.

Dual Model Architecture

Apple Intelligence runs both on-device and server-based models. These include:

- ❖ A foundation model for general tasks
- ❖ Specialized adapter models for things like summarization and tone adjustment Apple claims its server models match **GPT-4** performance, while its on-device models outperform similar small models from **Google and Microsoft.**

But before we conclude it is important for us to understand how iOS applications are developed. Yes! cause in the dynamic landscape of mobile technology, iOS app development stands as a cornerstone of innovation, user experience, and platform-specific optimization. Apple’s Integrated Development Environment (IDE), **Xcode**, serves as the central hub for building applications across **iPhone, iPad, Apple Watch, and Apple TV.**



The image above represents **Xcode** that is Apple's official IDE, offering a comprehensive suite of tools for designing, coding, testing, and deploying iOS apps. It includes:

- ❖ Interface Builder for designing user interfaces visually.
- ❖ Simulator for testing apps on virtual devices.
- ❖ Debugger and Profiler for performance analysis.
- ❖ Code Editor with intelligent suggestions and syntax highlighting.

Xcode supports both **Swift** and **Objective-C**, and integrates seamlessly with Apple's frameworks like **SwiftUI** and **UIKit**, enabling developers to build robust, responsive, and visually appealing applications.

Both Swift and Objective-C are programming languages that support iOS app development their differences lie as follow;

Swift

Swift is Apple's modern, safe, and fast programming language. It's designed for clarity and performance, making it ideal for both beginner and advanced developers. Key features include:

- ❖ Type safety and optional to reduce runtime errors.
- ❖ Functional programming capabilities.
- ❖ Seamless integration with **SwiftUI** and **Combine**.

Objective-C

Though largely replaced by **Swift**, Objective-C remains relevant for legacy codebases and certain low-level operations. It offers dynamic runtime features and is deeply embedded in Apple's older APIs.

Other languages like **Dart (Flutter)**, **JavaScript (React Native)**, and **C# (Xamarin)** are used in cross-platform development, but native iOS apps rely primarily on Swift and Objective-C.

Meanwhile frameworks that power iOS apps include;

SwiftUI

SwiftUI is a declarative framework introduced by Apple to simplify UI development. Developers describe what the UI should do, and SwiftUI handles the rendering and state updates. Benefits include:

- ❖ Real-time previews in Xcode.
- ❖ Cross-platform compatibility (**iOS, macOS, watchOS**).
- ❖ Built-in accessibility and localization support.

UIKit

UIKit is the imperative framework traditionally used for iOS development. It provides granular control over UI components and is preferred for complex, custom interfaces. UIKit supports:

- ❖ View controllers and navigation stacks.
- ❖ Gesture recognition and animations.
- ❖ Integration with Core Animation and Auto Layout.

Core ML, ARKit, and Combine

For advanced features, Apple offers specialized frameworks:

- ❖ Core ML for machine learning integration.
- ❖ ARKit for augmented reality experiences.
- ❖ Combine for reactive programming and data flow management.

The following video provides an in-depth look at some of the latest features and designs introduced in the iOS environment.

[Apple Ecosystem Explained: Features and Setup UPDATED for 2025!](#)

2.3.3. History and Evolution of the iOS Environment

The history of iOS is one of constant evolution, beginning in 2007 with the first iPhone and a revolutionary approach to mobile computing. It was not originally called iOS; it was known as iPhone OS, a simplified version of Apple's macOS. The OS was designed for a multi-touch screen, eschewing the physical keyboards and styluses common on other smartphones at the time.



But in **2025**, Apple's leap from **iOS 18 to iOS 26** wasn't a glitch in the matrix, it was a deliberate rebranding move to unify its entire ecosystem under a **year-based versioning system**. As seen below.



Question 3: But Why the Jump to iOS 26?

Well, Apple wanted to have several things aligned, which included:

- ❖ **Align version numbers across platforms:** Instead of having **iOS 18, macOS 14, and watchOS 10** all in the same year, Apple now uses “**26**” for all platforms that is the **iOS, iPadOS, macOS, watchOS, and even visionOS**.
- ❖ **Reflect the release cycle:** The number “**26**” corresponds to the **2025–2026 software season**, similar to how car models often use the upcoming year for branding.
- ❖ **Simplify user experience:** When users see “**26**” across all devices, it signals that everything is part of the same update family. Therefore, making it easier to understand and trust the continuity between devices.

Question 4: What's New in iOS 26?

The update isn't just cosmetic, it introduces a **Liquid Glass design language**, which:

- ❖ Adds dynamic, fluid UI elements across apps
- ❖ Enhances navigation, alerts, and controls with real-time responsiveness
- ❖ Offers customizable app icons and widgets with light/dark tinting
- ❖ Introduces spatial lock screen effects and adaptive time display.

Here's a crisp summary of the standout features.

Liquid Glass Design

- ❖ Translucent UI elements that reflect and refract content beneath them
- ❖ Dynamic tab bars and toolbars that morph based on context
- ❖ Clear and color-tinted app icons with layered glass effects
- ❖ Adaptive Lock Screen time display and spatial photo wallpapers

Apple Intelligence (AI Suite)

- ❖ **Live Translation:** Real-time translation in **Messages, FaceTime, and Phone** calls even spoken audio with AirPods.
- ❖ **Genmoji:** Create custom emoji by mixing existing ones or generating new ones from photos and expressions
- ❖ **Image Playground:** Generate images in any style using emoji prompts or ChatGPT

- ❖ **Siri + ChatGPT (GPT-5):** Siri now understands on-screen context and can act on ChatGPT responses

Visual Intelligence

- ❖ Ask questions about anything on your screen products, text, landmarks
- ❖ Summarize, translate, or read aloud screen content
- ❖ Create calendar events or search directly from screenshots

Music & Audio Enhancements

- ❖ **AutoMix:** DJ-style transitions between songs using beat matching
- ❖ Monthly replay insights and lyric pronunciation guides
- ❖ AirPods as remote shutter and high-def audio recorder

App Upgrades & UX Tweaks

- ❖ **Phone App:** AI-powered call screening and Hold Assist
- ❖ **Messages:** Spam filtering, chat backgrounds, and group polls
- ❖ **Safari:** Edge-to-edge design with floating tab bar
- ❖ **Camera:** Faster panoramic shots, lens-clean alerts, and spatial photo effects.

Privacy & Security

- ❖ Advanced Fingerprinting Protection across all browsing modes
- ❖ Simplified two-button navigation for accessibility.

i. The Early Years (2007–2009): A New Paradigm

- ❖ **iPhone OS 1 (2007):** The first version had no official name upon release. It was a groundbreaking OS that introduced **multi-touch gestures, a software keyboard, visual voicemail, and core applications like Safari, Mail, and Maps. It notably lacked an App Store and the ability to copy and paste.**
- ❖ **iPhone OS 2 (2008):** This version was a major turning point, **introducing the App Store.** This created a marketplace for third-party applications, which fundamentally transformed the mobile industry. It also added support for **Microsoft Exchange and push email, expanding its appeal to business users.**
- ❖ **iPhone OS 3 (2009):** This update addressed key user requests, **bringing features like copy and paste, MMS (multimedia messaging), and a system-wide search feature called Spotlight.**

ii. The Renaming and Major Overhauls (2010–2013)

- ❖ **iOS 4 (2010):** With the introduction of the iPad, Apple rebranded iPhone OS to iOS, signifying its expansion beyond just the iPhone. This version also brought **multitasking for the first time, along with FaceTime and app folders.**
- ❖ **iOS 5 (2011):** This update focused on wireless functionality, introducing iCloud for cloud-based synchronization and wireless activation for devices. It also launched **iMessage and a unified Notification Centre.**

- ❖ **iOS 6 (2012):** This was a controversial release due to Apple replacing Google Maps with its own mapping service. It introduced **Siri** out of beta, expanded its capabilities, and added features like **Passbook (now Wallet)**.
- ❖ **iOS 7 (2013):** Led by **Jony Ive**, this version marked a major visual and aesthetic overhaul. It replaced the skeuomorphic design with a flat, minimalist user interface. It also introduced the **Control Center** and **AirDrop**.

iii. The Modern Era (2014–Present)

- ❖ **iOS 8 & 9 (2014–2015):** These versions prioritized stability and introduced major features like **Apple Pay, Family Sharing, and HealthKit for tracking health data**.
- ❖ **iOS 10–13 (2016–2019):** These releases focused on improving core apps and services. Features included a revamped **Messages app with stickers and app extensions**, the **Files app**, and a **system-wide Dark Mode in iOS 13**. **iOS 13 also introduced a separate OS for iPads called iPadOS**.
- ❖ **iOS 14–17 (2020–2023):** This period brought significant personalization features, including **Home Screen widgets, the App Library, and App Tracking Transparency for enhanced privacy**.
- ❖ **iOS 18 (2024):** The latest stable release, iOS 18 continues to evolve the platform with features like **Apple Intelligence, a new Redesigned Control Center, and greater Home Screen customization**.

This video provides a deep dive into the history of iOS from its early days to its current state.

[History of iOS \(Full Documentary\)](#)

2.3.4. Advantages and Disadvantages of the iOS Environment

Advantages of the iOS Environment

- ❖ **Superior Security and Privacy:** iOS is widely regarded as a more secure platform due to its closed ecosystem. The App Store has a strict review process that acts as a strong barrier against malware and viruses. Additionally, iOS uses sandboxing to isolate apps from each other, and features like the Secure Enclave protect sensitive data like biometric information.
- ❖ **Optimized Performance and Stability:** By controlling both the hardware and software, Apple can tightly integrate the two. This ensures that the operating system runs smoothly and efficiently on every device, resulting in a fluid user experience with minimal crashes or glitches.
- ❖ **Consistent and Intuitive User Experience:** The iOS user interface is known for its simplicity and consistency. The design and layout of core features remain uniform across all devices, making it easy for users to learn and navigate. The gesture-based controls are polished and predictable.
- ❖ **Timely and Universal Software Updates:** Apple provides new OS updates and security patches to all supported devices simultaneously. This ensures that most users have the latest features and security protections promptly, reducing fragmentation and keeping the entire ecosystem secure.
- ❖ **Seamless Ecosystem Integration:** iOS devices work flawlessly with other Apple products. Features like Handoff allow you to continue a task on a different device,

while Airdrop makes sharing files between nearby Apple devices quick and effortless. This creates a cohesive and unified user experience.

Disadvantages of the iOS Environment

- ❖ **Closed and Restrictive Ecosystem:** The "**walled-garden**" model, while beneficial for security, limits user freedom. It's difficult to integrate with non-Apple products and services, and users are essentially locked into the ecosystem, making it challenging and often expensive to switch.
- ❖ **Limited Customization:** iOS offers minimal customization options compared to Android. **Users cannot change default apps, install custom launchers, or significantly alter the home screen layout.** This lack of personalization can be a drawback for users who prefer greater control over their device's appearance and functionality.
- ❖ **Higher Cost:** Apple's devices are generally more expensive than their Android counterparts with similar specifications. The higher price point can make the iOS ecosystem less accessible for budget-conscious consumers.
- ❖ **Restricted File Management:** The iOS file system is more limited and less accessible than on other platforms. Users cannot easily browse and manage files from different apps in a centralized location, which can be a point of frustration for those accustomed to a more open file system.

2.3.5. Differences between the Android OS and the iOS Environment

Listed below are major differences between the two mobile operating systems:

SN	FEATURE	ANDROID OS	iOS
1.	Ecosystem Model	Open-Source. The OS code is open to developers and manufacturers.	Closed-Source. The OS is proprietary and exclusive to Apple's hardware.
2.	Customization	High. Users can deeply customize the interface with launchers, widgets, and icon packs.	Low. Customization is limited to basic wallpaper and home screen layout changes.
3.	Device Diversity	Wide. Available on devices from countless manufacturers (Samsung, Google, etc.) across all price points.	Limited. Exclusively available on Apple's iPhone devices.
4.	The Store	Google Play Store. Has a larger volume of apps and is less restrictive on app publication.	App Store. Has a more curated collection with a strict app review process for quality and security.
5.	Security	Fragmentation. Updates are dependent on manufacturers and carriers, leading to inconsistent security.	Unified Control. Apple provides direct and timely updates to all supported devices.
6.	Software Updates	Delayed. Updates often take months or are never released for older devices.	Timely. Updates are released simultaneously to all supported devices for a unified experience.

7.	Cost	Diverse. Available on devices ranging from low-cost to premium.	High-End. Generally available on devices with a higher price point
8.	Parent Company	Google: Google develops the core Android software and makes the source code publicly available.	Apple: Apple is the sole developer and owner of iOS.
9.	CEO	Sundar Pichai: As the CEO of Google and its parent company, Alphabet , he oversees the strategic direction of Android.	Tim Cook: As the CEO of Apple, he continues to uphold the company's core principles of simplicity, security, and premium quality.

2.4. THE iOS ARCHITECTURE

The iOS architecture is a layered system, where lower layers provide fundamental services to higher layers and applications, abstracting away hardware details for developers. From bottom to top, the primary layers are the **Core OS Layer**, which provides low-level hardware services, the **Core Services Layer**, offering system-wide functionalities like networking and file access, the **Media Layer**, handling graphics, audio, and video. And the **Cocoa Touch Layer**, which

developers interact with to build user interfaces using frameworks like **UIKit**. As portrayed below.



i. Cocoa Touch Layer – User Interaction & App Behaviour

This is the topmost layer, where developers interact most directly. It provides the UI frameworks and event handling mechanisms.

COMPONENT	PURPOSE
UIKit	Core framework for building graphical, event-driven apps. Manages windows, views, controls, and gestures.
MapKit	Embeds maps and handles geolocation features within apps.
EventKitUI	UI components for accessing and displaying calendar events.
MessageUI	Enables sending emails and messages from within apps.
GameKit	Supports multiplayer gaming, leaderboards, achievements, and matchmaking.
PushKit	Manages push notifications, especially for VoIP and background updates

ii. Media Layer – Graphics, Audio, and Video

This layer handles all things visual and auditory, from rendering animations to playing sounds and videos.

COMPONENT	PURPOSE
Core Graphics	2D drawing engine for shapes, images, and text.
Core Animation	Smooth animations for UI elements and transitions.
OpenGL ES / GLKit	3D graphics rendering, often used in games and simulations.
Core Audio	Low-level audio processing and playback.
AV Foundation	High-level framework for working with time-based media (audio/video capture, editing, playback)

iii. Core Services Layer – Data, Networking, and System Services

This layer provides essential services like data storage, networking, and device capabilities.

COMPONENT	PURPOSE
Core Location	Tracks device location using GPS, Wi-Fi, and cell data.
Core Data	Object graph and persistence framework—great for managing app data.
CFNetwork	Low-level networking APIs for HTTP, FTP, and sockets.
HomeKit	Framework for communicating with smart home devices.
EventKit	Access to calendar and reminders data.
SystemConfiguration	Manages network settings and reachability.

iv. Core OS Layer – Low-Level Access & Security

This is the foundation of iOS, dealing with hardware interaction, security, and performance optimization.

COMPONENT	PURPOSE
Core Bluetooth	Communicates with Bluetooth-enabled devices.
Security Services	Handles encryption, certificates, and secure data storage.
External Accessory	Interfaces with external hardware accessories (e.g., fitness trackers).
Accelerate	High-performance math and DSP functions—great for scientific and image processing apps.
Local Authentication	Enables biometric authentication (Face ID, Touch ID).

v. Hardware Layer – The Physical Foundation

At the base is the device hardware which include the **CPU**, **GPU**, **sensors**, **radios**, *etc.* Which all software layers interact with via system calls and drivers.

MODULE 03

INTRODUCTION TO ANDROID APPLICATION PROGRAMMING USING ANDROID STUDIO

3.1. INTRODUCTION

Android programming is the process of creating software for devices that run on the Android operating system. This development is done primarily using either the **Java or Kotlin programming languages**, along with tools provided within the **Android Studio SDK**. It involves writing code, designing user interfaces, and ensuring the application works correctly on various Android devices.

An Android application is the final software product created through Android programming. These are the apps that users download from the **Google Play Store** or other sources and run them on their **Android smartphones, tablets, and other devices**. They can serve a wide variety of purposes, from **social media platforms and games to utility apps and business tools**.

Question 5: What is an SDK?

An SDK, or **Software Development Kit**, is a **set of tools, libraries, code samples, and documentation that developers use to create applications for a specific platform**. Think of it as a specialized toolbox that provides everything a developer needs to build an app, without having to start from scratch.

SDKs are essential because they abstract away the complexities of interacting with an operating system, hardware, or a third-party service. They provide pre-built components that simplify common tasks, allowing developers to focus on the unique features of their app.

A well-known example of an SDK is the **Android SDK**.

Android SDK

The Android SDK is the official toolkit provided by **Google** for building applications for the Android operating system. It includes:

- ❖ **Android Studio:** The **Integrated Development Environment (IDE)** used to write and debug code.
- ❖ **APIs (Application Programming Interfaces):** A set of rules and protocols that allow applications to interact with the Android OS and its features (e.g., the camera, GPS, or notifications).
- ❖ **Emulator:** A virtual device that allows developers to test their applications on different screen sizes and Android versions without needing a physical phone.
- ❖ **Debugging Tools:** Utilities for identifying and fixing errors in the application's code.

But wait a minute.....!

Question 6: Therefore, is there a difference between an SDK and an IDE?

An SDK and an IDE are both essential tools in software development, but they serve different purposes. The key difference is that **an SDK is a collection of tools to build an app**

for a specific platform, while an **IDE** is a single program that provides a workspace for a developer to write code.

SN	FEATURES	SDK (SOFTWARE DEVELOPMENT KIT)	IDE (INTEGRATED DEVELOPMENT ENVIRONMENT)
1.	Primary Purpose	Provides the tools and building blocks to create software for a specific platform.	Provides a workspace and a user interface to write and manage code.
2.	Analogy	A complete toolbox with all the necessary tools and instructions.	The workbench where the work is done and the tools are organized.
3.	Components	APIs, libraries, compilers, debuggers, documentation, and code samples.	Code editor, compiler, debugger, and build automation tools.
4.	Necessity	Required to build an application for a specific platform.	Optional (you can use a simple text editor and command-line tools), but highly recommended.

Imagine you're a skilled car mechanic tasked with building a specific type of car, say, a sports car.

The **SDK (Software Development Kit)** is the complete toolbox and parts kit for building that specific model. It contains all the specialized tools (**like a torque wrench for that specific engine**), a detailed blueprint (the API), and all the correct parts (the libraries and code samples) you need to assemble the car. You couldn't build the car without this specific kit.

The **IDE (Integrated Development Environment)** is your **workbench**. It's the physical workspace where you do all the work. It provides a clean, organized space with a place for your blueprints, a rack to hold your tools, and an easy-to-read checklist to guide your assembly process.

To build the car, you need both. You pull the tools and parts from your **SDK (the kit)** and use them on your **IDE (the workbench)**. The workbench makes the entire building process easier and more efficient, but the actual tools and parts are what allow you to create the final product.

3.1.1. Native Vs Cross-Platform Applications

A native application is built specifically for a **single platform** either **Android** or **iOS** using platform-specific programming languages and tools.

Tools & Technologies

- ❖ **Android:** Java, Kotlin, **Android Studio**
- ❖ **iOS:** Swift, Objective-C, **Xcode**

Key Characteristics

- ❖ **Performance:** Optimized for the device, offering fast and smooth operation
- ❖ **UI/UX:** Uses native components for a seamless user experience
- ❖ **Hardware Access:** Full access to device features like camera, GPS, sensors
- ❖ **Security:** Platform-specific encryption and security protocols

Native Application Examples

As explained earlier, these apps are built specifically for one platform (Android or iOS), using platform-specific languages like Kotlin or Swift. Examples include;

i. Android Native Apps

- ❖ **Google Maps:** Built with Kotlin/Java for deep integration with Android services
- ❖ **Spotify (Android version):** Optimized for Android's audio APIs and background services
- ❖ **WhatsApp (Android):** Uses native notifications, camera, and storage access

ii. iOS Native Apps

- ❖ **Apple Music:** Built with Swift, tightly integrated with iOS ecosystem
- ❖ **Safari:** Uses iOS-specific rendering and security features
- ❖ **Instagram (iOS version):** Originally native for iOS before expanding

These apps benefit from **high performance, smooth UI, and full access** to device features.

Meanwhile on the other end, a cross-platform application is developed using **a single codebase that runs on multiple platforms (e.g., Android, iOS, Windows).**

Tools & Technologies

FRAMEWORK	LANGUAGE	PLATFORMS SUPPORTED
Flutter	Dart	Android, iOS, Web, Desktop
React Native	JavaScript	Android, iOS
Xamarin	C#	Android, iOS, Windows
Unity	C#	Android, iOS, Windows, more (mostly for games)

Key Characteristics

- ❖ **Code Reusability:** One codebase for multiple platforms
- ❖ **Faster Development:** Reduces time and cost
- ❖ **UI Consistency:** May not fully match native UI guidelines
- ❖ **Performance Trade-offs:** Slightly less optimized than native apps.

Cross-Platform Application Examples

As equally explained above, these are built using frameworks like **Flutter, React Native, or Xamarin**, allowing a single codebase to run on multiple platforms.

i. Flutter Apps

- ❖ **Google Ads:** Built with Flutter for Android and iOS
- ❖ **Reflectly:** A journaling app with beautiful UI across platforms.
- ❖ **Alibaba:** Parts of the app use Flutter for consistent UX.

ii. React Native Apps

- ❖ **Facebook:** Uses React Native for parts of its mobile experience
- ❖ **Instagram:** Some features are cross-platform via React Native
- ❖ **Skype:** Unified codebase for Android and iOS

iii. Xamarin Apps

- ❖ **Storyo:** A storytelling app using C# and Xamarin
- ❖ **The World Bank app:** Cross-platform access to global data.

These apps are ideal for startups, MVPs, or educational tools where speed and cost-efficiency matter.

Question 7: Therefore, what are the key differences between Native Applications and Cross-Platform Applications?

FEATURE	NATIVE APP	CROSS-PLATFORM APP
Codebase	Separate for each platform	Single shared codebase
Performance	High (optimized for platform)	Moderate to high (depends on framework)
Development Time	Longer	Shorter
Cost	Higher (multiple teams/tools)	Lower (shared resources)
UI/UX	Platform-specific, polished	May vary across platforms
Tooling	Android Studio, Xcode	Flutter, React Native, Xamarin
Use Case	High-performance apps (e.g., banking, gaming)	MVPs, startups, internal tools

3.2. How to Download and Install Android Studio on Windows

Before we actually dive deep into the download and whole installation process of Android Studio it's good for each and every one of us to know what this IDE is as well as its components, features and more.

Android Studio is the official Integrated Development Environment (IDE) for developing applications for the Android operating system. It is a single, comprehensive application that provides developers with all the tools they need to write, test, debug, and package Android apps. Based on **JetBrains' IntelliJ IDEA**, Android Studio is designed to streamline the entire development workflow.



Android Studio

3.2.1. Key Components of Android Studio

Android Studio's power comes from its integration of several specialized components into one unified environment.

i. Code Editor

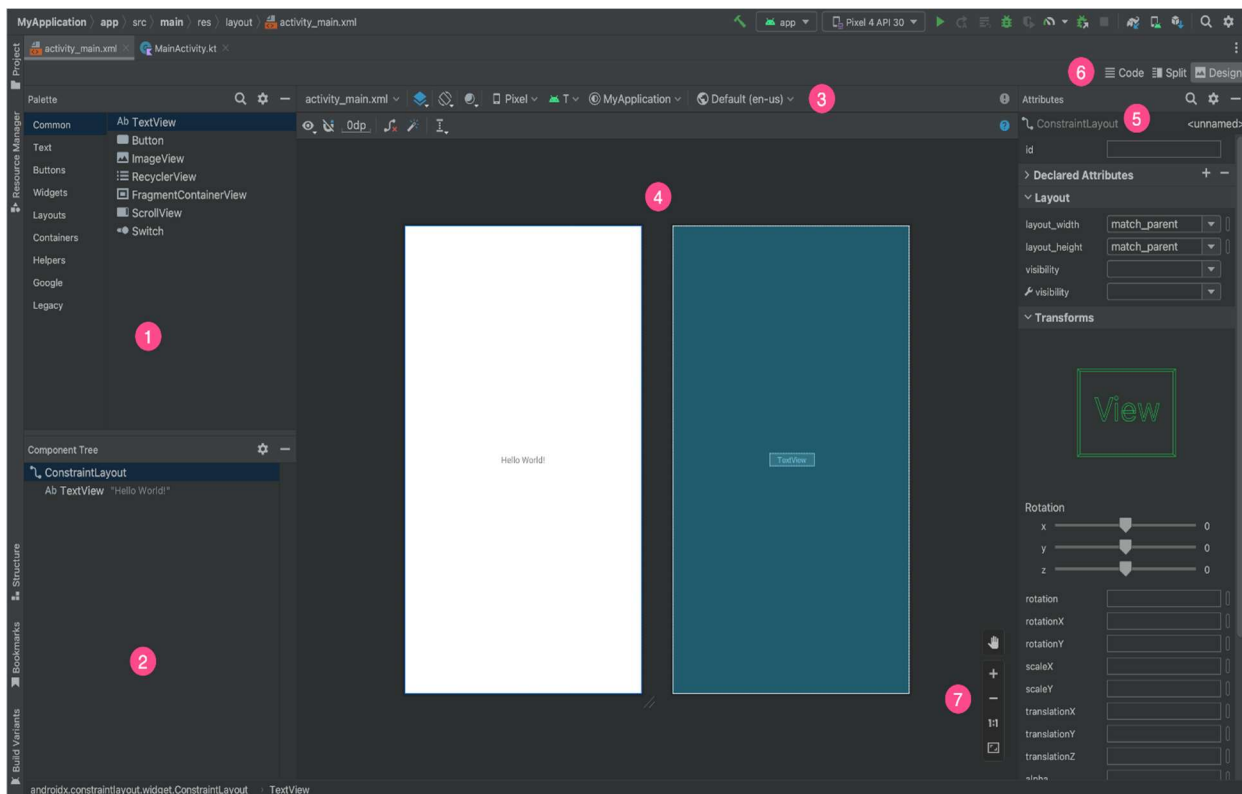
The intelligent code editor is the central workspace where developers write code. It supports multiple languages, including Kotlin (**the preferred language for Android**), Java, and C++. It provides smart features that boost productivity, such as:

- ❖ **Syntax Highlighting:** Makes code more readable by color-coding different elements.
- ❖ **Intelligent Code Completion:** Predicts and suggests code as you type, helping to reduce errors and speed up development.
- ❖ **Code Refactoring:** Tools that allow for restructuring code without changing its external behaviour.

ii. Layout Editor

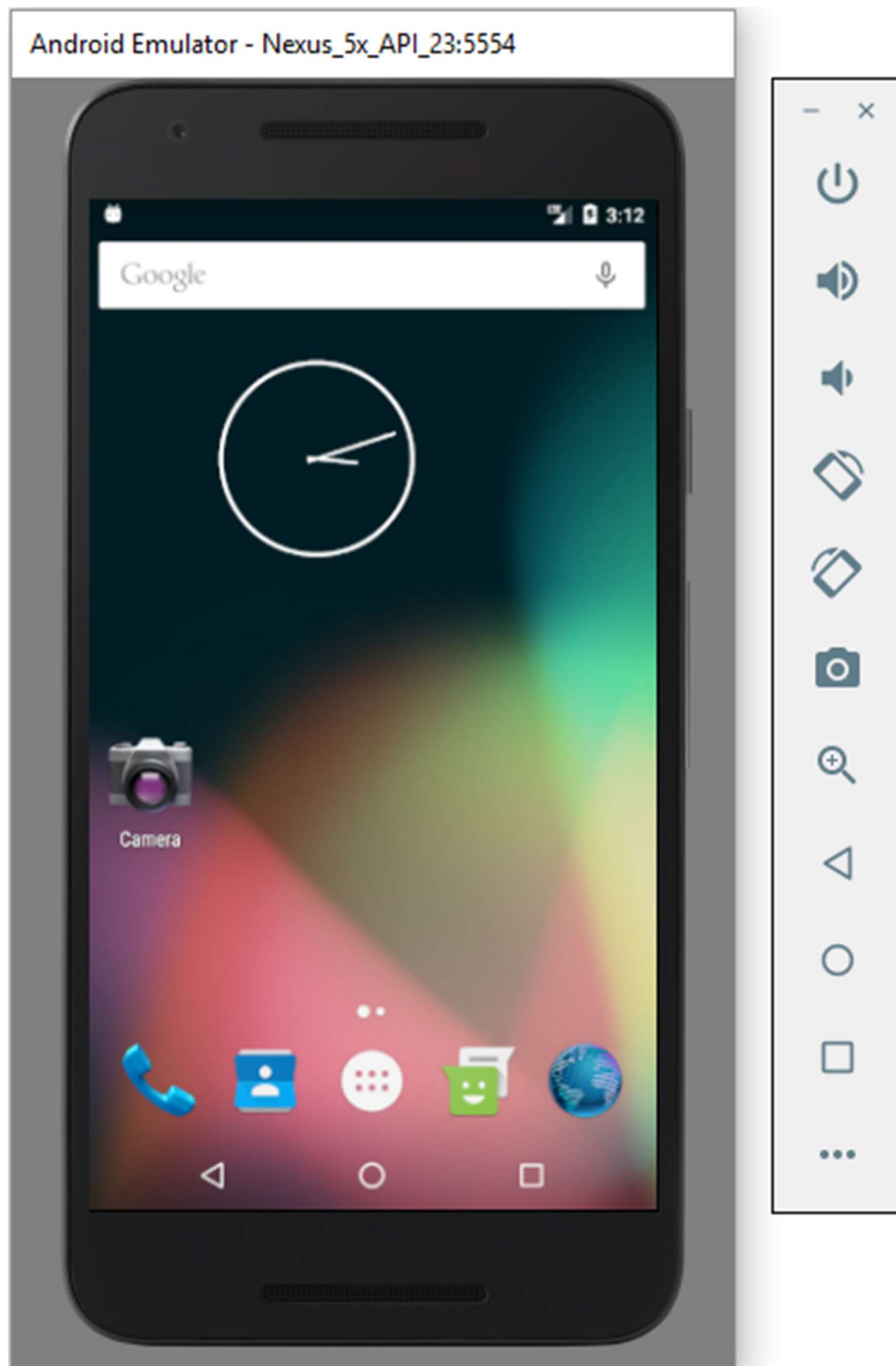
The **Layout Editor** is a **visual design tool for building an app's User Interface (UI)**. Instead of writing complex XML code by hand, developers can use a drag-and-drop interface to place UI elements like buttons, text fields, and images onto a screen. This component also provides a real-time preview of the layout on various virtual devices, helping developers ensure their app looks correct on different screen sizes and resolutions.

As shown below



iii. Android Emulator

The **Android Emulator** is a virtual device that simulates a real Android phone on a computer. It allows developers to test their applications on a wide variety of devices (different phones, tablets, and Android versions) without needing to own the physical hardware. The emulator can simulate features like **screen rotation**, **GPS location**, and **network speeds**, providing a comprehensive testing environment.



Question 8: In respect to android programming what is an AVD and how is it different from an Emulator?

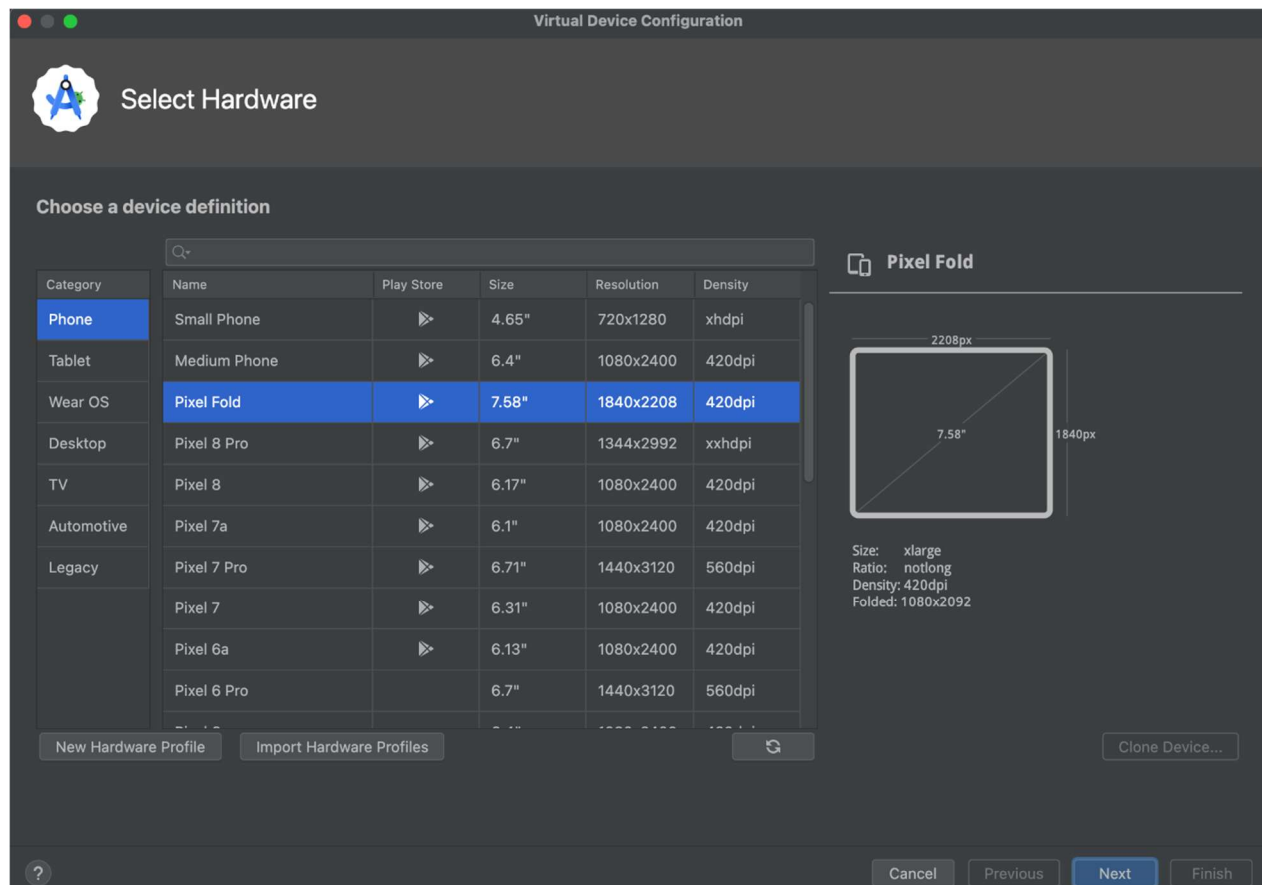
In Android development, understanding the difference between an AVD and an Emulator is like understanding the difference between a **blueprint** and the **house** built from it. While people often use the terms interchangeably, they refer to different parts of the virtualization process.

An **AVD** which stands for *Android Virtual Device* is a *configuration file that defines the specific characteristics of a device you want to simulate*. It acts as a "blueprint" for a virtual phone or tablet. Each AVD includes:

- ❖ **Hardware Profile:** Defines features like *RAM size, screen resolution, camera availability, and sensors (accelerometer, GPS)*.
- ❖ **System Image:** Specifies the version of the Android OS (e.g., *Android 14, Android 11*) and the CPU architecture (*x86 or ARM*) it will run.
- ❖ **Storage & Skin:** Manages dedicated user data storage and the visual look (*frame*) of the device on your screen.

Meanwhile on the other hand, an **Emulator** is *the software engine (an independent application) that actually executes the AVD*. It mimics the hardware and software features of a real mobile device on your development machine.

Therefore, we can conclude by saying this one can have many AVDs, but one can only use one Emulator to run whichever AVD he/she needs to test at that moment. Setting up an AVD looks like this.



iv. Gradle Build System

Gradle is the automated build system that Android Studio uses. **Its role is to take all the source code files, resources, and dependencies and compile them into a finished, ready-to-install Android application package (APK or AAB).** Gradle handles complex tasks like dependency management (downloading and linking required libraries), multi-module projects, and creating different versions of an app (e.g., a debug version and a release version).

Unlike older build tools, **Gradle uses a Domain Specific Language (DSL)** based on **Groovy or Kotlin (KTS)** to define project configurations, making it highly flexible and scriptable. Gradle plays a major role because it manages the entire lifecycle of the software build. Its core responsibilities include:

- ❖ **Dependency Management:** If your app needs a library for networking (like Retrofit) or image loading, you don't manually download a .jar file. You simply list the library in the build.gradle file, and Gradle handles the rest.
- ❖ **Build Variants:** It allows you to create different versions of your app from the same codebase (e.g., a "free" version with ads and a "pro" version without).
- ❖ **Manifest Merging:** It takes your AndroidManifest.xml and merges it with manifests from the libraries you are using to create one final, comprehensive manifest.
- ❖ **Resource Processing:** It compiles your XML layouts and optimizes your images to ensure they take up as little space as possible.
- ❖ **Code Minification:** Using tools like **R8/ProGuard**, Gradle can shrink your code and remove unused resources to reduce the final app size.

Question 8: Why does Gradle need an Internet Connection to Sync?

When you click **"Sync Project with Gradle Files"** in Android Studio, you often see a progress bar that requires an internet connection. This is necessary for several reasons:

- ❖ **Downloading remote dependencies** since most Android libraries are hosted on remote repositories like **Maven Central**, **Google's Maven Repository**, or **JCenter**.
- ❖ **Resolving dependency graphs**, here we must understand that libraries often depend on other libraries (transitive dependencies). Gradle connects to the internet to download "metadata" (POM files) to understand the entire tree of required code. It ensures that there are no version conflicts (e.g., if two libraries require different versions of the same core component).
- ❖ **Downloading the Gradle Wrapper**, Android projects use a **"Gradle Wrapper."** If you open a project that requires a version of Gradle you don't have installed, the wrapper will automatically download the correct version of the Gradle distribution from the internet to ensure the build is consistent across different developer machines.
- ❖ **Plugin Updates**, The **Android Gradle Plugin (AGP)** which adds Android-specific build capabilities to the base Gradle tool frequently requires updates or specific versions that must be fetched from Google's servers.

When you add implementation **"com.google.android.material:material:1.9.0"** to your project, Gradle must go online to find that specific version, download the library files, and cache them on your computer.

Note: Once a "Sync" is successful, the files are cached locally on your machine. You can then use "Offline Mode" in Android Studio to build your project without an internet connection, provided you don't add any new dependencies or change your Gradle version.



3.3. Create Your First Android App

Here is a simple code for creating a simple Android Application to display a text that says "Hello World". But I strongly recommend you to edit the code and try to print your own name.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!" />

</LinearLayout>
```

MainActivity.kt

```
package name
import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
    }
}
```

```
}  
}
```



3.3.1. Layouts Basics

A layout defines the visual structure for a user to interface with your app, such as in an activity. Android provides a range of libraries, canonical starting points, and techniques to display and position content.

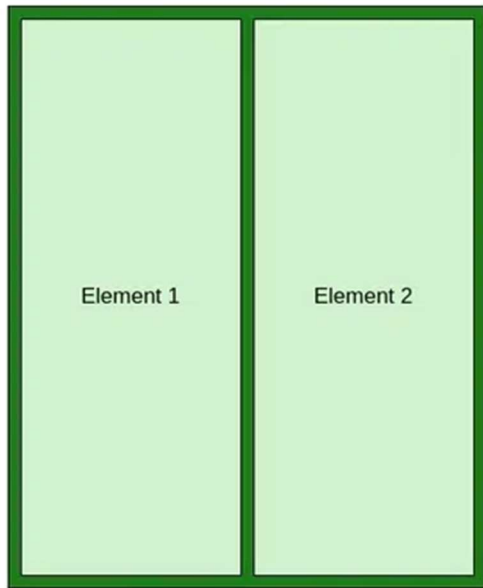
Layouts are responsible for defining the structures and arrangements of elements in an application. It helps to organize views in a hierarchy, allowing for responsive and flexible design. Some common types are mentioned below.

- a. **LinearLayout** in Android is a ViewGroup subclass, used to arrange child view elements one by one in a singular direction either horizontally or vertically based on

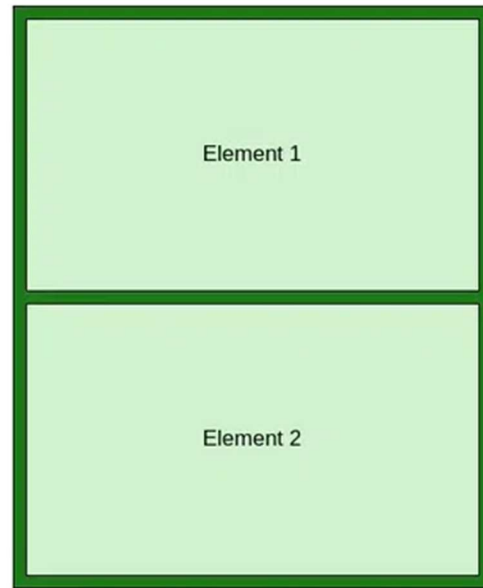
the orientation attribute. We can specify the linear layout orientation using the **android:orientation** attribute.

All the child elements arranged one by one in either multiple rows or multiple columns. As illustrated with the Horizontal and Vertical List with an Image.

- i. **Horizontal list:** One row, multiple columns.
- ii. **Vertical list:** One column, multiple rows.



Horizontal Linear Layout



Vertical Linear Layout

Question 9: How Can One Declare LinearLayout in an XML file?

Here's an example of how to declare a LinearLayout in an XML file

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <!--sub view 1-->
    <View
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <!--sub view 2-->
    <View
        android:layout_width="wrap_content"
```

```

        android:layout_height="wrap_content" />

<!--sub view 3-->
<View
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />

</LinearLayout>

```

We are going to do so in two-steps, first is adding `LinearLayout` in our `activity_main.xml` below, is a sample code which includes a **TextView**, an **EditText** and a **Button** in **Vertical orientation**.

`activity_main.xml`

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp"
    tools:context=".MainActivity">

    <!-- sub-view 1 :TextView -->
    <TextView
        android:id="@+id/txtVw"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Name:"
        android:layout_margin="16dp"
        android:textStyle="bold"
        android:textSize="24sp" />

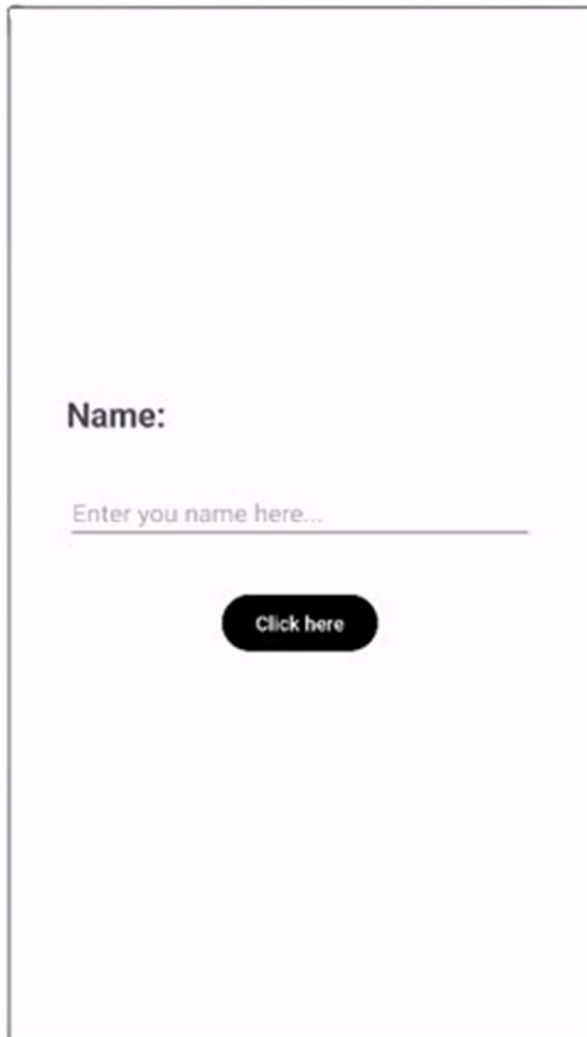
    <!-- sub-view 2 :EditText -->
    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:autofillHints="name"
        android:hint="Enter you name here... "
        android:inputType="text"/>

    <!-- sub-view 3 :Button -->
    <Button
        android:id="@+id/showInput"
        android:layout_width="wrap_content"

```

```
android:layout_height="wrap_content"  
android:backgroundTint="@color/black"  
android:layout_margin="16dp"  
android:text="Click here"  
android:textColor="@color/white" />  
</LinearLayout>
```

Design UI



Step 2 once the layout is created, you can access the UI elements from the `activity_main.xml` file by using the `findViewById()` method in the `onCreate()` call back function in the `MainActivity.kt` file.

```
package com.example.text_view_application  
  
import androidx.appcompat.app.AppCompatActivity  
import android.os.Bundle  
import android.widget.Button
```



```
import android.widget.EditText
import android.widget.TextView
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.text_view_application.ui.theme.Text_View_ApplicationTheme

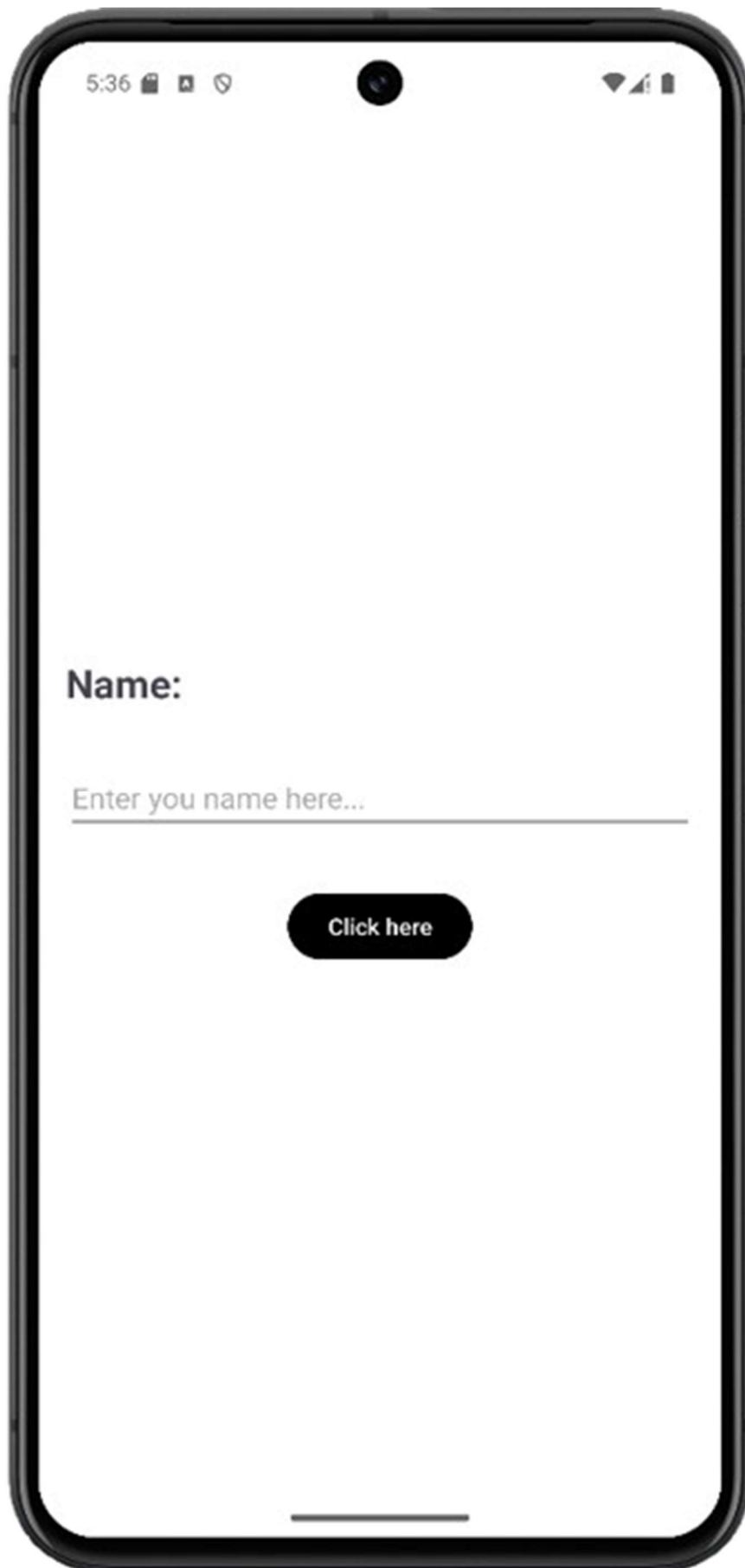
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        val showButton: Button = findViewById(R.id.showInput)

        val editText: EditText = findViewById(R.id.editText)

        val textView: TextView = findViewById(R.id.txtVw)
    }
}
```

After running the application, this is the expected result from the above code:



5:36 [Battery Icon] [Signal Icon] [Wi-Fi Icon]

Name:

Enter you name here...

Click here

- b. **Relative Layout** in Android is a layout that arranges its child views in relation to each other. Unlike Linear Layout, which can make element arrangement complex, RelativeLayout simplifies the process by allowing flexible and dynamic positioning. It enables you to align views relative to each other or to the edges of the parent container, making it ideal for creating more complex and responsive UI designs.

There are so many properties that are supported by relative layouts. Some of the most used properties are listed below:

- ❖ `layout_alignParentTop`
- ❖ `layout_alignParentBottom`
- ❖ `layout_alignParentRight`
- ❖ `layout_alignParentLeft`
- ❖ `layout_centerHorizontal`
- ❖ `layout_centerVertical`
- ❖ `layout_above`
- ❖ `layout_below`

Benefits of Using RelativeLayout

- ❖ **Reduces nesting:** RelativeLayout avoids the need for deeply nested layouts, which can help improve the performance of the app.
- ❖ **More flexible:** It allows for precise control over the layout of views and is useful in responsive designs.

Here is a step-by-step implementation of how working with RelativeLayout looks like.;

- ❖ **Step 1:** Create a New Project
- ❖ **Step 2:** Navigate to the `app > res > layout > activity_main.xml` and add the below code to that file.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="#168BC34A"
    xmlns:android="http://schemas.android.com/apk/res/android">

    <Button
        android:id="@+id/button1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="35sp"
        android:text="Top Left Button"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true"/>

    <Button
        android:id="@+id/button2"
```

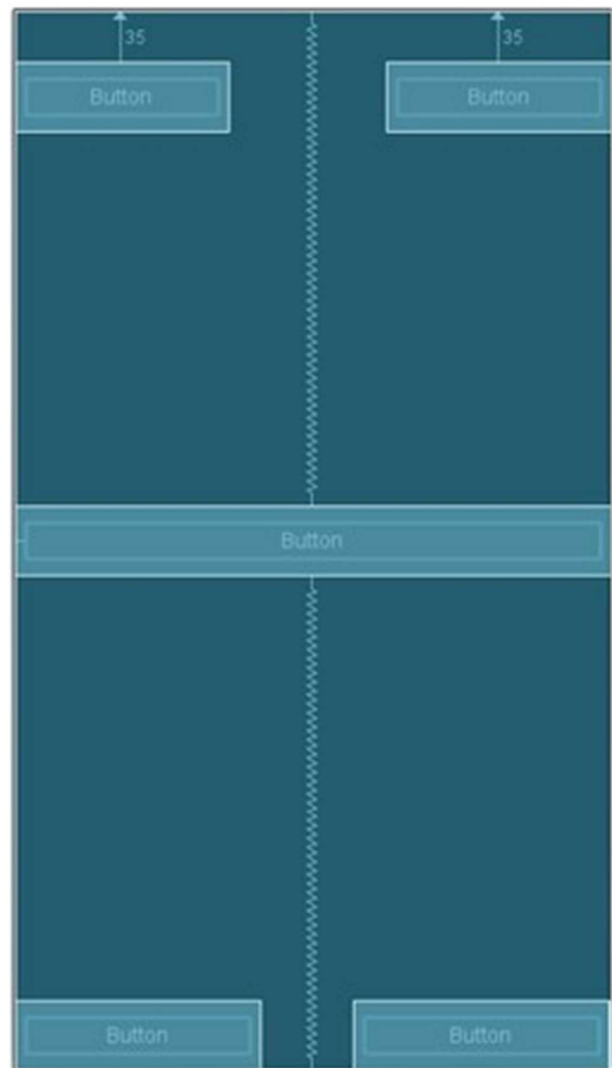
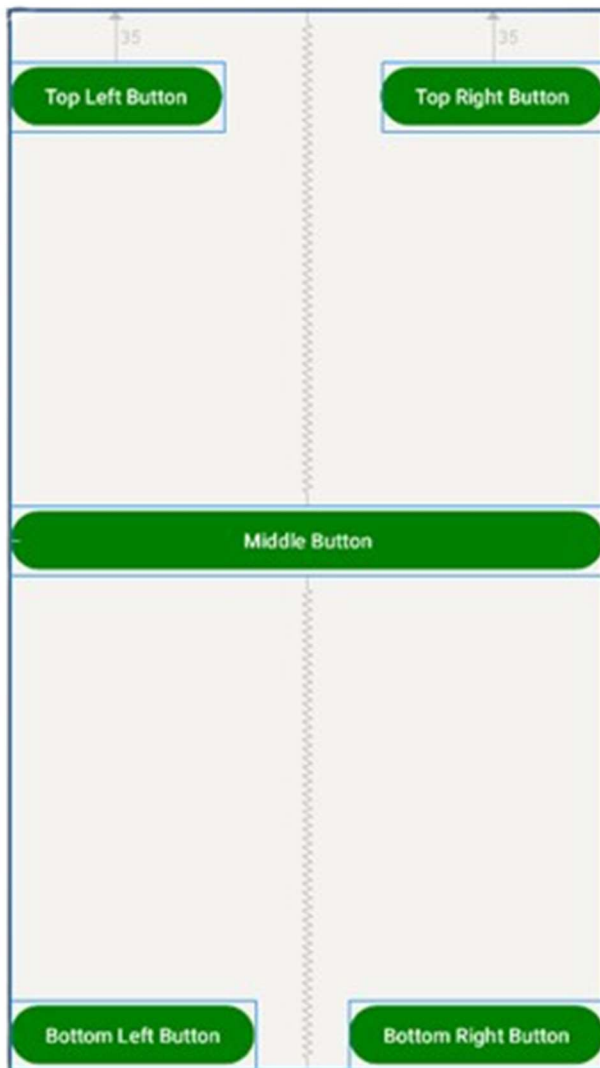
```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="35sp"
        android:text="Top Right Button"
        android:layout_alignParentTop="true"
        android:layout_alignParentRight="true"/>

<Button
    android:id="@+id/button3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Bottom Left Button"
    android:layout_alignParentLeft="true"
    android:layout_alignParentBottom="true"/>

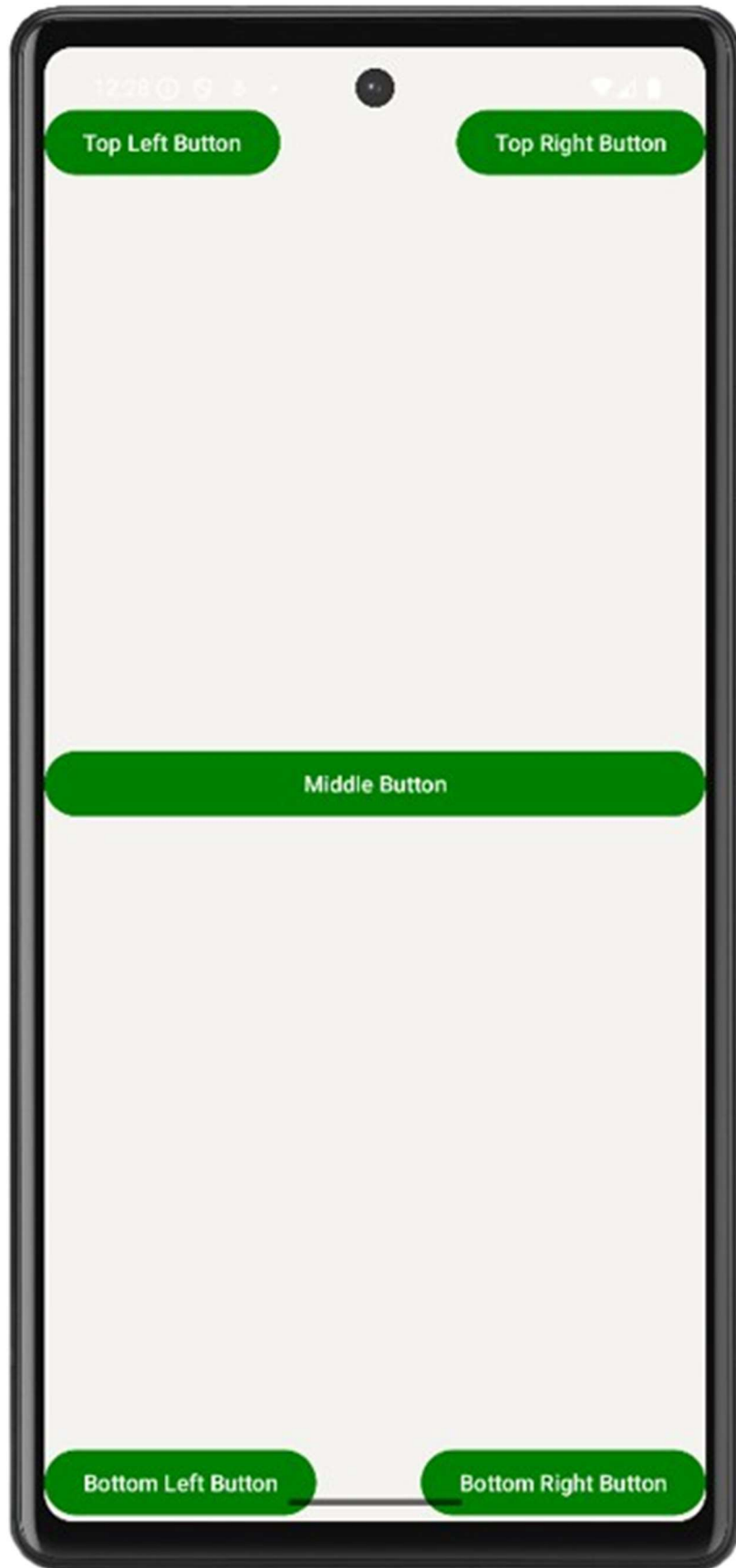
<Button
    android:id="@+id/button4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Bottom Right Button"
    android:layout_alignParentRight="true"
    android:layout_alignParentBottom="true"/>

<Button
    android:id="@+id/button5"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:text="Middle Button"
    android:layout_centerVertical="true"
    android:layout_centerHorizontal="true"/>
</RelativeLayout>
```

UI Design



Since there is nothing extraordinary to add in our MainActivity file, well all's what left to do is to run our app to have displayed the below output.



Before moving any further, First, we have written an xml open tag which contains xml version and encoding type.

- ❖ In the next line, we have written an open tag for relative layout.
- ❖ Inside the relative layout, we have specified layout width, height, and orientation.
- ❖ Then we have created 5 buttons which are arranged at **topRight, topLeft, bottomRight, bottomLeft, bottomRight, and center** respectively.
- ❖ Finally, we have closed the relative layout tag.
- ❖ In the kotlin code, there are just import statements.
- ❖ And in the main class, there is an **onCreate()** method. Which is used to create the activity.
- ❖ Activity is just a user interface.

3.3.2. Components or Views Basics

Views are UI components used to display content or handle user interactions, such as **buttons, text fields, and images**. They are the building blocks of the user interface, allowing developers to design layouts and define app behaviour.

a. TextView

A **Widget** refers to the elements of the UI (User Interface) that helps the user interact with the Android App. A **TextView** is one of many such widgets which can be used to improve the UI of the app. TextView refers to the widget which displays some text on the screen based on the **layout, size, colour and more** set for that particular TextView. It optionally allows us to modify or edit itself as well.

XML Attributes of TextView in Android

Attributes	Description
android:text	Sets text of the Textview
android:id	Gives a unique ID to the Textview
android:cursorVisible	Use this attribute to make cursor visible or invisible. Default value is visible.
android:drawableBottom	Sets images or other graphic assets to below of the Textview.
android:drawableEnd	Sets images or other graphic assets to end of Textview.
android:drawableLeft	Sets images or other graphic assets to left of Textview.
android:drawablePadding	Sets padding to the drawable (images or other graphic assets) in the Textview.
android:autoLink	This attribute is used to automatically detect url or emails and show it as clickable link.
android:autoText	Automatically correct spelling errors in text of the Textview.
android:capitalize	It automatically capitalizes whatever the user types in the Textview.
android:drawableRight	Sets drawables to right of text in the Textview.
android:drawableStart	Sets drawables to start of text in the

	Textview.
android:drawableTop	Sets drawables to top of text in the Textview.
android:ellipsize	Use this attribute when you want text to be ellipsized if it is longer than the Textview width.
android:ems	Sets width of the Textview in ems.
android:gravity	We can align text of the Textview vertically or horizontally or both.
android:height	Use to set height of the Textview.
android:hint	Use to show hint when there is no text.
android:inputType	Use to set input type of the Textview. It can be Number, Password, Phone etc.
android:lines	Use to set height of the Textview by number of lines.
android:maxHeight	Sets maximum height of the Textview.
android:minHeight	Sets minimum height of the Textview.
android:maxLength	Sets maximum character length of the Textview.
android:maxLines	Sets maximum lines Textview can have.
android:minLines	Sets minimum lines Textview can have.
android:maxLength	Sets maximum width Textview can have.
android:minWidth	Sets minimum lines Textview can have.
android:textAllCaps	Show all texts of the Textview in capital letters.
android:textColor	Sets color of the text.
android:textSize	Sets font size of the text.
android:textStyle	Sets style of the text. For example, bold, italic, bolditalic.
android:typeface	Sets typeface or font of the text. For example, normal, sans, serif etc
android:width	Sets width of the TextView.

Below is the code of the **layout file** and the **Kotlin file** of how a **TextView** is implemented given below.

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:gravity="center">

    <TextView
        android:id="@+id/textview"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome To The"
        android:textSize="26sp"
        android:textStyle="bold"
        android:textColor="@color/colorPrimary"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</RelativeLayout>
```

The **MainActivity** file responsible to adding functionalities to the layout.

```
package com.example.swe
import android.os.Bundle
import android.view.View
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
    // Creating the instance of the TextView created
    private var textView: TextView? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Linking the activity file to this Kotlin file
        setContentView(R.layout.activity_main)

        // Get the TextView with the id
        // mentioned in the layout file
        textView = findViewById<View>(R.id.textview) as TextView

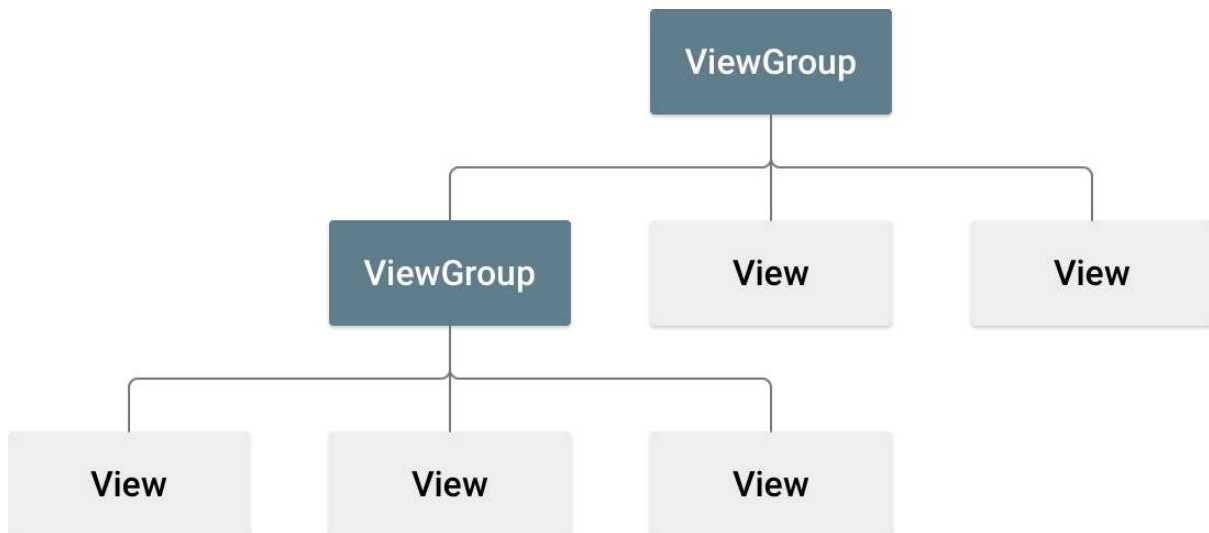
        // Try to change the text of the Textview upon touch
        // and also display a Toast message
```

```
textView!!.setOnClickListener { // Changing text
    textView!!.text = "SWE II FAMILY"

    // Displaying Toast message
    Toast.makeText(this@MainActivity, "Welcome To The SWE II FAMILY",
        Toast.LENGTH_SHORT).show()
}
```

3.3.2.1. Layouts in Views

All elements in the layout are built using a hierarchy of **View** and **ViewGroup** objects. A **View** usually draws something the user can see and interact with. A **ViewGroup** is an invisible container that defines the layout structure for **View** and other **ViewGroup** objects, as shown in the image below.



View objects are often called **widgets** and can be one of many subclasses, such as **Button** or **TextView**. The **ViewGroup** objects are usually called layouts and can be one of many types that provide a different layout structure, such as **LinearLayout** or **ConstraintLayout**.

You can declare a layout in two ways

- ❖ Declare UI elements in XML. Android provides a straightforward XML vocabulary that corresponds to the **View** classes and subclasses, such as those for widgets and layouts. You can also use Android Studio's Layout Editor to build your XML layout using a drag-and-drop interface.
- ❖ Instantiate layout elements at runtime. Your app can create **View** and **ViewGroup** objects and manipulate their properties programmatically.

Declaring your UI in XML lets you separate the presentation of your app from the code that controls its behaviour. Using XML files also makes it easier to provide different layouts for **different screen sizes and orientations**.

Question 8: What is the difference between the `LinearLayout` and the `ConstraintLayout`?

`LinearLayout` arranges views in a single direction either vertically or horizontally meanwhile, **`ConstraintLayout`** allows flexible positioning of views relative to each other and the parent, making it more powerful for complex UI designs.

Key Differences Between `LinearLayout` and `ConstraintLayout`**i. Layout Structure**

- ❖ **`LinearLayout`:** Arranges child views in a straight line (either vertical or horizontal). It possesses a simple and predictable layout behaviour. An example includes *a login form with fields stacked vertically*.
- ❖ **`ConstraintLayout`:** Positions views using constraints relative to other views or the parent. It supports complex layouts in a flat hierarchy. An example includes *a responsive dashboard with overlapping elements and dynamic alignment*.

ii. Hierarchy and Performance

- ❖ **`LinearLayout`:** May require nested layouts for complex designs, increasing view hierarchy depth, which may lead to slower rendering and layout inflation.
- ❖ **`ConstraintLayout`:** Reduces the need for nested layouts by allowing all views in a single layer, thus, improving performance and simplifying layout management.

iii. Flexibility

- ❖ **`LinearLayout`:** Limited to one axis; harder to create responsive or adaptive designs, thus making it ideal for simple forms or lists.
- ❖ **`ConstraintLayout`:** Highly flexible; supports guidelines, barriers, chains, and bias positioning therefore making it ideal for adaptive UIs across screen sizes and orientations.

iv. Ease of Use

- ❖ **`LinearLayout`:** Easier for beginners to understand and implement, as it has less configurations needed.
- ❖ **`ConstraintLayout`:** Steeper learning curve but more powerful. Android Studio provides visual tools to simplify constraint setup.

Write the XML

Using Android's XML vocabulary, you can quickly design UI layouts and the screen elements they contain, in the same way that you create web pages in HTML with a series of nested elements.

Each layout file must contain exactly one root element, which must be a **View or ViewGroup object**. After you define the root element, you can add additional layout objects or widgets as child elements to gradually build a View hierarchy that defines your layout. For example, here's an XML layout that uses a vertical **`LinearLayout`** to hold a **`TextView`** and a **`Button`**:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">

    <Button
        android:id="@+id/btnShowMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text=""
        android:textSize="18sp"
        android:layout_marginTop="20dp" />
</LinearLayout>
```

Now in **MainActivity.kt (Kotlin Code)** set up the button click listener:

```
package com.example.helloworld

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnShowMessage = findViewById<Button>(R.id.btnShowMessage)
        val tvMessage = findViewById<TextView>(R.id.tvMessage)

        btnShowMessage.setOnClickListener {
            tvMessage.text = "Hello World!"
        }
    }
}
```

Now what happens is once the user sees a button labeled **"Click Me"**. Once the button is clicked, the **TextView** updates to show **"Hello World!"**.

Question 8: What is the Android Manifest file?

The **AndroidManifest.xml** is an essential XML file found in the root of the **main source** set in every Android application project. It acts as the *"identity card" or "blueprint"* for the app. It is the essential control file and blueprint for every Android application.

It acts as a contract between your application and the Android operating system (OS). Before the system can launch any of your app's code, it must read the Manifest file to understand the app's structure, requirements, and capabilities. It's good for us to equally understand that the Android Operating System (OS) and the Google Play Store use this file to understand how your app is structured, what it can do, and what it needs to run before any code is executed. Here is a detailed breakdown of its core roles and components:

i. Declaring App Components

The manifest must list the four fundamental components of an Android app. If a component is not declared here, the OS cannot run it.

- ❖ **Activities:** The screens of your app (*e.g., Login Screen, Home Screen*).
- ❖ **Services:** Background tasks (*e.g., playing music*).
- ❖ **Broadcast Receivers:** Listeners for system-wide events (*e.g., battery low, screen on*).
- ❖ **Content Providers:** Mechanisms for sharing data (*e.g., contacts*) between apps.

ii. Permissions

This is where you tell the OS what sensitive data or hardware features your app needs to access.

- ❖ **Examples:** Accessing the internet, camera, location (GPS), or reading contacts.
- ❖ When a user installs (or runs) your app, the system checks this file to prompt the user to grant these permissions.

iii. Hardware and Software Requirements

It specifies the types of devices your app is compatible with.

- ❖ **Hardware:** Does your app *require* a camera or GPS? If so, the Play Store will hide your app from devices that don't have those features.
- ❖ **Software:** It defines the minimum Android version (API level) required to run the app.

iv. App Metadata

It defines the visible properties of the application on the device.

- ❖ **Icon:** The image displayed on the home screen.
- ❖ **Label:** The name of the app shown to users.
- ❖ **Theme:** The visual style (colors, fonts) used throughout the app.

v. Entry Point

It tells the Android launcher which Activity to start when the user taps the app icon. This is done using an `<intent-filter>` with the action `MAIN` and category `LAUNCHER`.

```
<manifest ...>

<uses-permission android:name="android.permission.INTERNET" />

<application

    android:icon="@mipmap/ic_launcher"

    android:label="@string/app_name">

    <!-- This is the entry point (Splash Screen or Home Page) -->

    <activity android:name=".MainActivity" android:exported="true">

        <intent-filter>

            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />

        </intent-filter>

    </activity>

</application>

</manifest>
```

As mentioned above, this *AndroidManifest.xml* file acts as the "identity card" for an Android app, telling the operating system everything it needs to know before running the app.

Here is a line-by-line explanation of what this specific code does:

Permissions

```
<uses-permission android:name="android.permission.INTERNET" />
```

This line explicitly asks the Android system for permission to access the Internet. **But why?** Well, by default, Android apps cannot access the network for security reasons. If your app needs to fetch data from a server or load a webpage, you **must** include this line. If you forget it, your app will crash or fail to load data.

Application Attributes

```
<application  
  
    android:icon="@mipmap/ic_launcher"  
  
    android:label="@string/app_name">
```

- ❖ **android:icon:** This points to the image file (**usually located in the res/mipmap folder**) that will be used as the app's icon on the user's home screen.
- ❖ **android:label:** This points to a string (located in res/values/strings.xml) that determines the name of the app as it appears under the icon.

Activity Declaration

```
<activity android:name=".MainActivity" android:exported="true">
```

- ❖ **android:name=".MainActivity":** This registers a specific screen (Activity) with the system. The **.** (**dot**) indicates that the file **MainActivity.kt** is located in the current package.
- ❖ **android:exported="true":** This is a security setting. Setting it to true allows external components (**like the Android home screen launcher**) to start this Activity.

The Intent Filter (The Entry Point)

```
<intent-filter>  
  
    <action android:name="android.intent.action.MAIN" />  
  
    <category android:name="android.intent.category.LAUNCHER" />  
  
</intent-filter>
```

This block is crucial. It tells Android exactly how to start this specific Activity.

- ❖ **android.intent.action.MAIN:** This tells the system, "This is the main entry point of the application." It is the first thing that should run.
- ❖ **android.intent.category.LAUNCHER:** This tells the system, "Put an icon for this Activity in the phone's app launcher list."

Together, the **<intent-filter>** block ensures that when a user taps your app's icon on their phone, the MainActivity screen is the one that opens up. Without this block, the app would be installed, but there would be no icon to click on to open it!

In short, we can conclude by saying that the **Android Manifest file** is the most important configuration file in an Android project. It is the single point of truth that the operating system consults to determine the app's identity, its components, its security requirements, and its compatibility standards before it allows any of the application's Java or Kotlin code to execute.

b. EditText

EditText refers to the widget that displays an empty text field in which a user can enter the required text and this text is further used inside our application.

Here the layout can be any layout like Relative, Linear and more and the attributes can be many among the table given below.

Below is the implementation of the above approach:

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/white"
    android:gravity="center"
    android:padding="64dp"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/text_view_id"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:autofillHints="name"
        android:inputType="text"
        android:hint="Enter your name..." />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/button_id"
        android:layout_width="match_parent"
        android:layout_height="56dp"
        android:layout_marginTop="32dp"
        android:text="Submit"
        android:textColor="@color/white"
        android:backgroundTint="@color/green"/>

</LinearLayout>
```

MainActivity.kt

```

package com.example.editttext

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    // Declare UI elements: EditText for user input and Button for submission
    private lateinit var editText: EditText
    private lateinit var button: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Link the EditText and Button variables to their UI elements in activity_main.xml
        editText = findViewById(R.id.text_view_id);
        button = findViewById(R.id.button_id);

        // Set up an onClickListener for the button to handle click events
        button.setOnClickListener {
            val name = editText.text.toString();

            // Display a short message (Toast) showing a welcome message with the user's input
            Toast.makeText(this, name, Toast.LENGTH_SHORT).show()
        }
    }
}

```

Some Important XML Attributes of EditText in Android

Attributes	Description
android:id	Used to uniquely identify the control
android:gravity	Used to specify how to align the text like left, right, center, top, etc.
android:hint	Used to display the hint text when text is empty
android:text	Used to set the text of the EditText
android:textSize	Used to set size of the text.
android:textColor	Used to set color of the text.
android:textStyle	Used to set style of the text. For example, bold, italic, bolditalic, etc.
android:textAllCaps	Used this attribute to show the text in capital letters.

android:width	It makes the TextView be exactly this many pixels wide.
android:height	It makes the TextView be exactly this many pixels tall.
android:maxWidth	Used to make the TextView be at most this many pixels wide.
android:minWidth	Used to make the TextView be at least this many pixels wide.
android:background	Used to set background to this View.
android:backgroundTint	Used to set tint to the background of this view.
android:clickable	Used to set true when you want to make this View clickable. Otherwise, set false.
android:drawableBottom	Used to set drawable to bottom of the text in this view.
android:drawableEnd	Used to set drawable to end of the text in this view.
android:drawableLeft	Used to set drawable to left of the text in this view.
android:drawablePadding	Used to set padding to drawable of the view.
android:drawableRight	Used to set drawable to right of the text in this view.
android:drawableStart	Used to set drawable to start of the text in this view.
android:drawableTop	Used to set drawable to top of the text in this view.
android:elevation	Used to set elevation to this view.

c. **ImageView**

ImageView class is used to display any kind of image resource in the android application either it can be **android.graphics.Bitmap** or **android.graphics.drawable.Drawable** (It is a general abstraction for anything that can be drawn in Android). ImageView class or **android.widget.ImageView** inherits the **android.view.View** class which is the subclass of Kotlin. AnyClass.Application of ImageView is also in applying tints to an image in order to reuse a drawable resource and create overlays on background images. Moreover, ImageView is also used to control the size and movement of an image.

Whenever **ImageView** is added to an activity, it means there is a requirement for an image resource. Thus, it is obvious to provide an Image file to that ImageView class. It can be done by adding an image file that is present in the Android Studio itself or we can add our own image file. Android Studio owns a wide range of drawable resources which are very common in the android application layout. The following are the steps to add a drawable resource to the ImageView class.

When you define an **ImageView** in your layout file (*activity_main.xml*), you typically use these attributes:

- ❖ **android:src:** This is the most important attribute. It specifies the image to be displayed. You usually point to an image in the res/drawable folder (e.g., *@drawable/my_logo*) or an icon in res/mipmap.
- ❖ **android:scaleType:** This determines how the image should be resized or moved to fit within the ImageView boundaries. Common values include:
 - **centerCrop:** Resizes the image to fill the view, cropping the edges if necessary (very common for profile pictures).
 - **fitCenter:** Resizes the image to fit inside the view while maintaining its aspect ratio.
 - **center:** Displays the image at its original size in the center.
 - **android:contentDescription:** Essential for accessibility. It provides a text description of the image for screen readers used by visually impaired users.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">

<ImageView
    android:id="@+id/imageView5"
    android:layout_width="0dp"
    android:layout_height="361dp"
    android:layout_marginStart="14dp"
    android:layout_marginTop="153dp"
    android:layout_marginEnd="14dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:srcCompat="@drawable/le_ninja" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

You can also change or manipulate the image dynamically while the app is running:

MainActivity.kt

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
```



```
{  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
}  
}
```

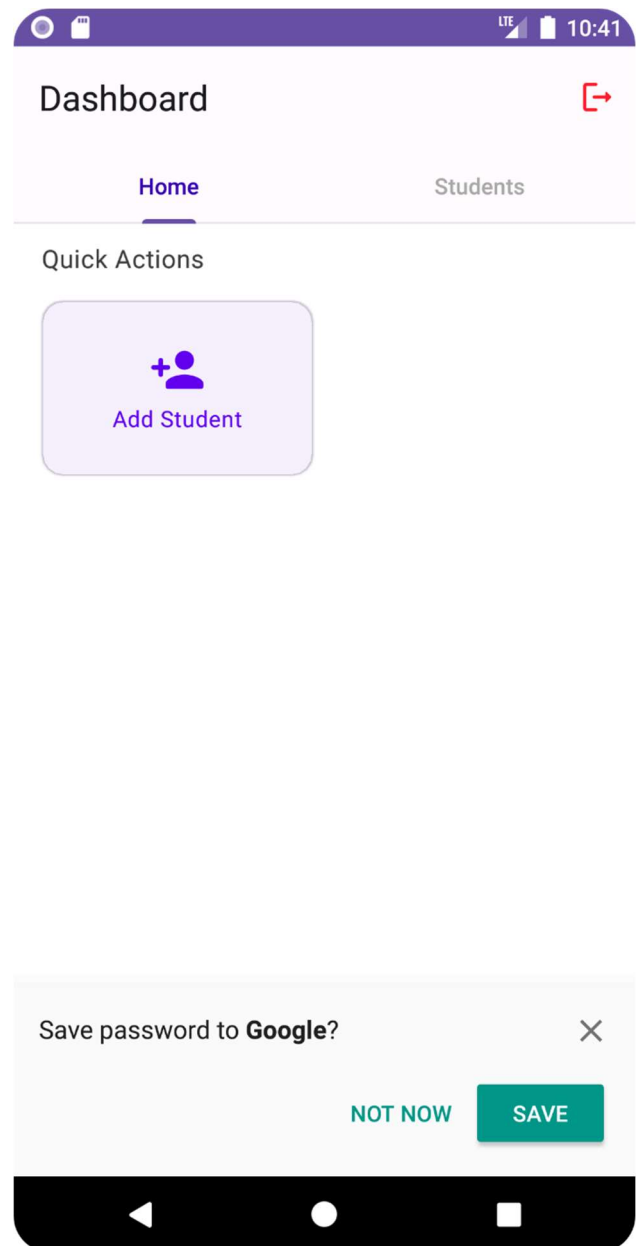
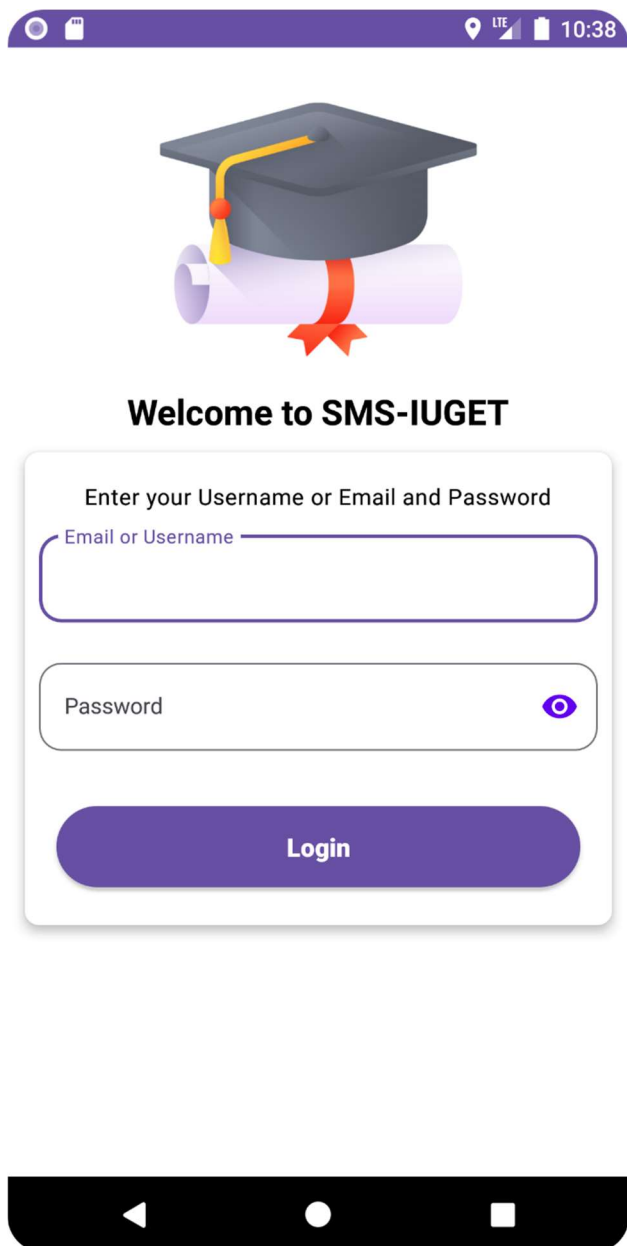
Vector vs. Raster

- ❖ **Raster (PNG/JPG):** These are fixed-pixel images. If you stretch them too much, they get blurry.
- ❖ **Vector (Vector Asset):** Android Studio allows you to import SVGs as "Vector Assets." These are mathematical descriptions of shapes, meaning they look perfectly sharp at any size (ideal for icons).

Tinting

A cool feature of **ImageView** is the **android:tint** attribute. This allows you to apply a color "filter" over an image. This is extremely useful for making black icons appear in different colors (like blue or white) without needing to create separate image files for each color.

Exercise 1: Design and Implement a Student Management System connecting it to any relational database management system of your choice either it being MySQL and SQLite as samples here are images of how yours can look like.



Dashboard 

Home **Students**

DA Djamien Abigayil  

Save password to **Google**? 

NOT NOW **SAVE**

← **New Student** **Add**

Personal Info

First name

Last name

Date of birth

☒ Male

☐ Female

Academic Info

Field of study

Degree

☐ Level 1

10:42

← New Student Add

Personal Info

First name
Christ

Last name
Jerry

D
2

Operation successful

OK

Academic Info

Field of study
CNSM

Degree
Btech

Level 1

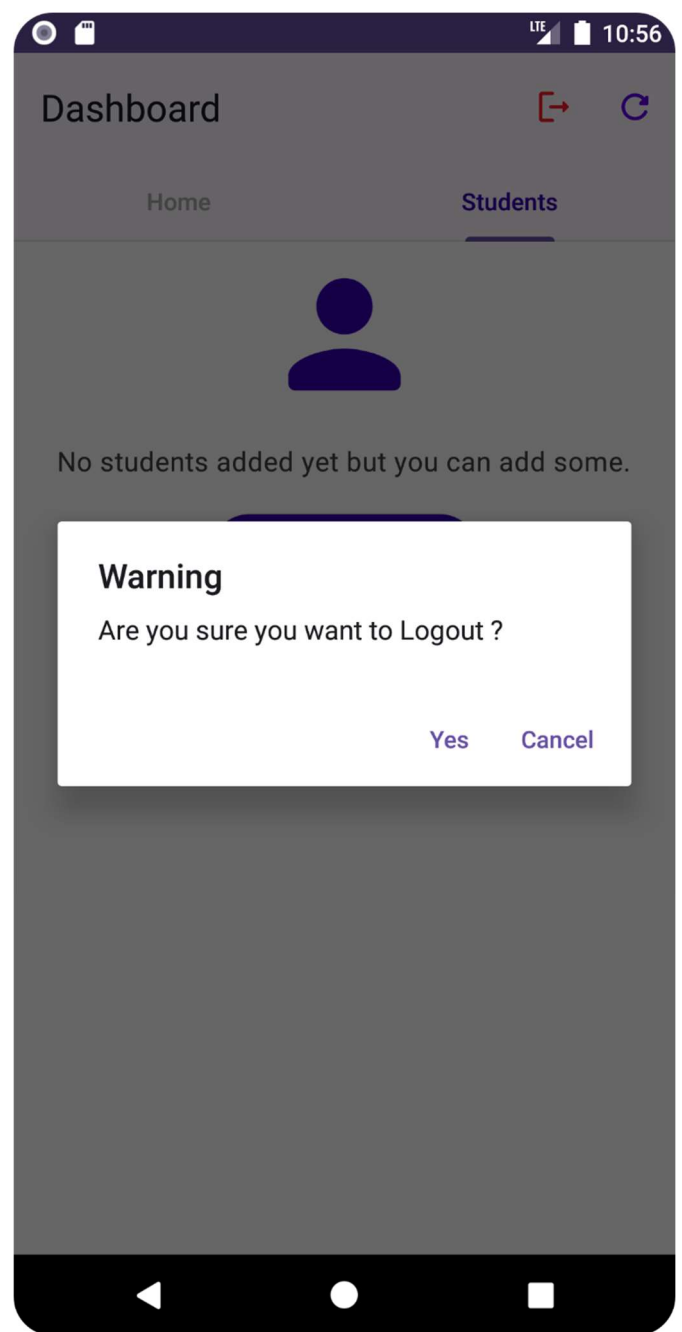
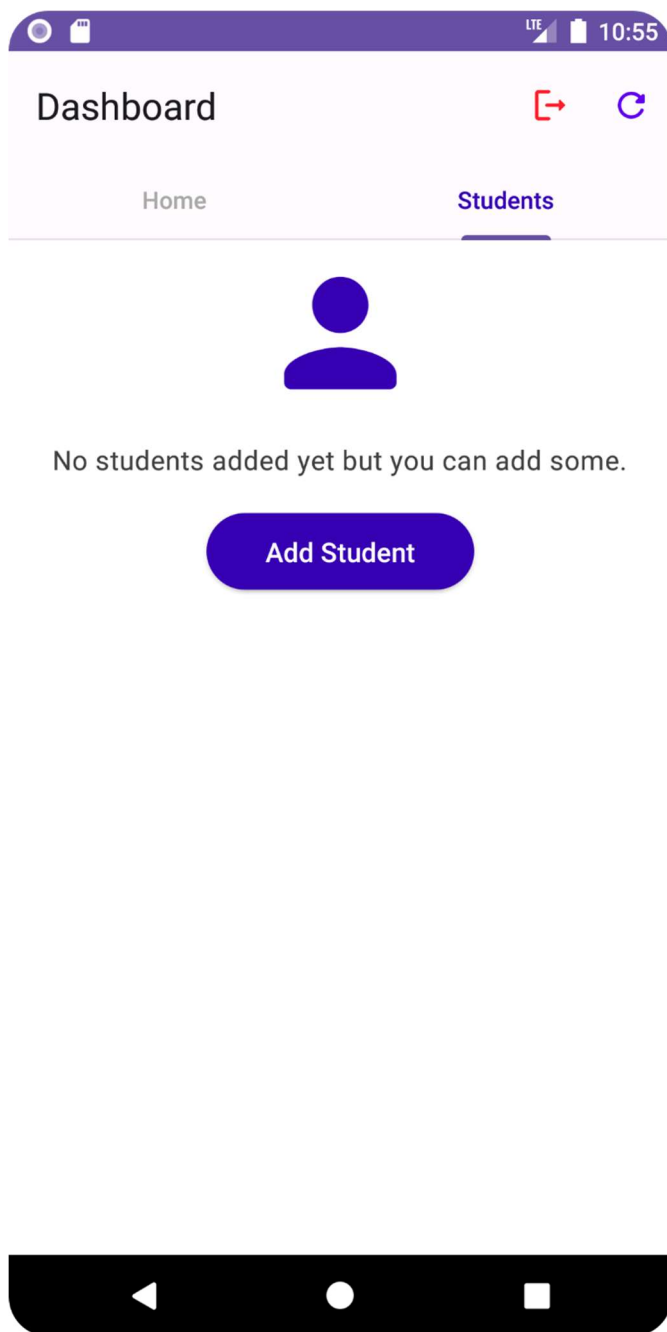
10:52

Dashboard

Home Students

DA Djamien Abigayil

CJ Christ Jerry



3.4. Android Application Core Components

The Android operating system organizes mobile applications around **four** primary, fundamental building blocks known as the Android Application Core Components. Every application is built using some combination of these four components.

i. Activity

An Activity represents a single screen with a user interface. It's where users interact with the app. But essentially, what we can retain from all these is that, **an activity is a screen in your application.** For example, an email application might have an **"Inbox Activity,"** a **"Compose Email Activity,"** and a **"Settings Activity."**

As a key feature, activities are managed by the **Activity Stack (or back stack).** When a new Activity starts, it is pushed onto the top of the stack. When the user presses the Back button, the top Activity is popped off, and the previous Activity resumes.

Activities have a specific lifecycle (**e.g., onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy()**) that developers must manage to handle how the application behaves during user navigation or system events (like a phone call coming in)

Question 8: What are all six callback methods used for?

These methods are critical for managing the state and resources of an application's visual screens (Activities) as the user navigates. These six callback methods collectively define the **Activity Lifecycle**, which the Android system uses to manage an activity's state from the time it is launched until it is shut down.

Here is a detailed explanation of what each method is used for:

a) onCreate()

This is the very first method called when the Activity is first created. You perform basic application startup logic here, which should only happen once for the entire life of the Activity.

Key Tasks

- ❖ Initialize the user interface by calling **setContentView(R.layout.activity_main).**
- ❖ Initialize necessary variables and class-scope objects.
- ❖ Set up event handlers and bindings.
- ❖ Retrieve persistent data that might be saved from previous sessions.

b) onStart()

Called when the Activity is about to become visible to the user. It handles tasks that need to be performed just before the UI is shown. The Activity is visible but is not yet in the foreground (it cannot receive user input yet).

Key Tasks

- ❖ Initialize code that maintains the UI (**like registering a BroadcastReceiver**) or other resource management that is needed while the Activity is visible.

c) onResume()

Called when the Activity has come into the foreground and is ready to interact with the user. This is the state where the app is fully interactive. The Activity moves into this state when it is created for the first time, or when it returns from an onPause() state (**e.g., the user receives a call and then returns to the app**).

Key Tasks

- ❖ Start animations, music playback, or anything that requires real-time updates and user interaction.
- ❖ Re-acquire exclusive resources like camera access or GPS.

d) onPause()

Called when the system is about to start another Activity (**which will be visible on top of this one**) or when the user navigates away from the Activity but it is still partially visible (**like when a dialog pops up**). To save user progress and release system resources that shouldn't run while the Activity is paused. This method should execute very quickly.

Key Tasks

- ❖ Stop animations or other CPU-intensive work.
- ❖ Commit unsaved changes to permanent storage (e.g., save the state of a game or text entry).
- ❖ Release resources like camera or sensor handlers to allow other apps to use them.

e) onStop()

Called when the Activity is no longer visible to the user (**e.g., the user switches to a different app or presses the Home button**). To perform more extensive, resource-intensive cleanup that is not strictly necessary in onPause().

Key Tasks

- ❖ Release larger resources that are not needed while the UI is hidden (e.g., unregister a listener).
- ❖ The Activity object still exists in memory, but it's stopped and hidden.

f) onDestroy()

The final call before the Activity instance is completely destroyed and removed from system memory. This can happen for two reasons:

- ❖ The user explicitly finishes the Activity (by pressing Back in the final screen).
- ❖ The system is temporarily destroying the Activity to reclaim resources (e.g., due to a configuration change like a screen rotation).

To perform final cleanup and release all remaining resources.

Key Tasks

- ❖ Tear down all resources associated with the Activity (**threads, cursors, etc.**).
- ❖ It is crucial to clean up here to prevent memory leaks.

ii. Services

A **Service** is a component that runs in the background to perform long-running operations or to supply functionality to other applications, without providing a user interface. Services are ideal for tasks that shouldn't interrupt the user's current activity, such as playing music in the background, fetching data over the network, or monitoring sensor input.

Types of Services

- ❖ **Started Service:** A service that runs a job in the background and stops itself when done (e.g., **downloading a large file**).
- ❖ **Bound Service:** A service that offers an interface for other application components (or other apps) to interact with it (e.g., **a service that provides a spell-checking utility**).

iii. Broadcast Receivers

A **Broadcast Receiver** is a component that allows the app to listen for system-wide broadcast announcements (events) and respond to them.

These components act as "**listeners**" for events that are triggered by the system or by other applications. Examples of broadcasts include "**battery low**," "**Wi-Fi enabled**," "**screen turned off**," or "**a picture was taken**."

A Broadcast Receiver typically does very minimal work, often just launching a Service or displaying a Notification to react to the event.

iv. Content Providers

A **Content Provider** manages a shared set of application data. They are the standard interface for sharing data between different applications.

They are the mechanism Android uses for secure and structured data sharing. For instance, the system's contacts or calendar data is managed by a Content Provider.

Here is a key feature, if your app needs to access data from another app (**like selecting a contact's phone number**), it uses the other app's Content Provider. Similarly, if you want to expose data from your app for others to use, you would implement a Content Provider. They abstract the actual data storage mechanism (e.g., **an SQLite database, flat files, or the network**) from the consuming application.

Question 9: What is a process in an android application?

In an Android application, a process refers to the Linux-based execution environment that runs the app's code.

Here's a breakdown of what a process is;

The Linux Foundation

Android is built on top of the Linux kernel. In Linux, a process is an instance of a program being executed. Every Android application, by default, runs in its own unique Linux process.

Default Process Management (Isolation)

When you launch an app:

- ❖ **Dedicated Process:** The system creates a new Linux process for that application.
- ❖ **Unique VM:** Within that process, the system starts a dedicated instance of the ART (Android Runtime) the virtual machine that executes the app's code (which is compiled from Java/Kotlin into DEX bytecode).
- ❖ **Isolation and Security:** This isolation is crucial for security and stability. If one application's process crashes, it won't take down the entire system or other running applications.

The Main Thread (UI Thread)

Every process starts with a single thread of execution, often called the main thread or UI thread.

- ❖ **UI Responsibility:** The main thread is responsible for handling all user interface updates, drawing the screen, and responding to user input (like touches and gestures).
- ❖ **The Golden Rule:** Because of this, you must not perform long-running operations (like network calls, database queries, or large file processing) on the main thread. If you block the main thread for too long, the application becomes unresponsive, leading to the infamous ANR (Application Not Responding) error.

Application Components and Processes

The core components of an Android app (**Activity**, **Service**, **BroadcastReceiver**, **ContentProvider**) usually all run within the same default process.

However, an application can ask the system to run certain components (**specifically Services**) in separate, distinct processes by defining the **android:process** attribute in the **AndroidManifest.xml**.

- ❖ **Multi-Process Applications:** While rare for simple apps, running components in separate processes can be useful for:
- ❖ **Resource Management:** Isolating a memory-intensive task so that if that process is killed due to memory pressure, the main UI process remains active.
- ❖ **Inter-Process Communication (IPC):** Allowing different parts of your application (or even different applications) to communicate securely.

Question 10: Meanwhile on the other hand what's a thread?

If a Process is the container and owner of resources (like memory space, open files, etc.), a Thread is the actual sequence of execution within that process.

Here is a detailed breakdown of what a thread is:

Definition of a Thread

A thread is the smallest unit of execution that can be scheduled by the operating system.

- ❖ **Execution Path:** It is a single, sequential flow of control within a program. Think of it as a single to-do list the CPU is working through.
- ❖ **Shared Resources:** All threads within the same process share the process's resources, including its memory space, code, and global data. This sharing makes communication and data exchange between threads very fast (but also makes them harder to manage safely).

Concurrency and Multithreading

The primary reason for using multiple threads (multithreading) is concurrency.

In an Android application:

- ❖ **The Main Thread (UI Thread):** As mentioned, this is the single thread responsible for drawing the UI and handling user input. It must run smoothly, ideally completing any task in under 16 milliseconds to maintain a **60 frames per second (fps)** experience.
- ❖ **Background Threads:** Any task that takes a noticeable amount of time (like network requests, file reading, image decoding, or heavy calculations) must be moved off the Main Thread and run on a background thread (also called a worker thread).

By using a background thread, the application achieves concurrency:

- ❖ The Main Thread remains free to handle UI events (keeping the app responsive).
- ❖ The Background Thread performs the heavy lifting (like downloading a file).

Once the background thread finishes its work, it must safely pass the result back to the Main Thread, as only the Main Thread is allowed to modify the user interface elements (Views).

The Relationship Between Process and Thread

FEATURE	PROCESS	THREAD
Identity	An instance of an application running. The "container."	A single execution path within a process. The "worker."
Resources	Owns its own memory space, system resources, and files.	Shares memory and resources with sibling threads.
Communication	Inter-Process Communication (IPC); slower, more complex.	Direct access to shared memory; fast, but requires synchronization.
Minimum	A process must have at least one thread (the main thread).	A thread must belong to a process.

In simple terms:

- ❖ **Process:** The entire shop, with its own building and supplies.
- ❖ **Thread:** A dedicated worker inside that shop, carrying out a specific set of instructions.

If you only had one worker (thread) trying to take orders and bake the bread and clean the floor, the whole shop would stop when the worker bakes bread. By adding more workers (threads), the shop (process) can do all those things concurrently.

3.5. The Kotlin Programming Language

Kotlin is a modern, trending programming language that was released in **2016** by **JetBrains**. Which has built world-class IDEs like **IntelliJ IDEA**, **PhpStorm**, **Appcode**, and **more**. It was first introduced by JetBrains in **2011** as a new language for the JVM. Kotlin is equally an object-oriented programming language, and way better than Java, but still fully interoperable with Java code. It has seen massive adoption, primarily because **Google** designated it as the preferred language for **Android application development in 2017**.



It has become very popular since it is compatible with Java which means that Java code (and libraries) can be used in Kotlin programs.

3.5.1. Kotlin Syntax Basics

At a basic level the entry point of a Kotlin application is the main function:

```
fun main() {  
    println("Hello world!")  
}
```

One noticeable difference with Kotlin is how variables are been declared using these any of the two keywords of **val (Immutable)** & **var (Mutable)**. Whereby, **val** cannot be changed after assignment. Meanwhile, **var** can be reassigned.

```
// Declares the variable x and initializes it with the value of 5  
val x: Int = 5  
// 5
```

```
// Declares the variable x and initializes it with the value of 5
var x: Int = 5
// Reassigns a new value of 6 to the variable x
x += 1
// 6
```

Read from the standard input

The **readln()** function reads from the standard input. This function reads the entire line the user enters as a string.

You can use the **println()**, **readln()**, and **print()** functions together to print messages requesting and showing user input:

```
// Prints a message to request input
println("Enter any word: ")

// Reads and stores the user input. For example: SWE II ROCKS
val yourWord = readln()

// Prints a message with the input
print ("You entered the word: ")
print(yourWord)
// You entered the word: SWE II ROCKS
```

3.5.2. Key Features of Kotlin

As Kotlin's key features are what make it such a popular and highly-regarded modern language. Below are the most important features that distinguish Kotlin, focusing on conciseness, safety, and modern development practices:

i. Null Safety

This is the most celebrated feature, designed to eliminate the notorious runtime error known as the **NullPointerException (NPE)**.

- ❖ **Non-nullable by Default:** Variables cannot hold a null value unless you explicitly allow it using the safe call operator (?). For example: *var name: String cannot be null. var name: String? can be null.*
- ❖ **Safe Call Operator (?.):** Allows you to call a method or access a property only if the object is not null. If the object is null, the entire expression evaluates to null, avoiding a crash.
- ❖ **The Elvis Operator (?:):** Provides a concise way to specify a default value if an object is null. For example: *val length = text?.length ?: 0 (If text is null, use 0 for length).*

ii. Conciseness and Reduction of Boilerplate

Kotlin significantly reduces the amount of code required compared to Java, making programs easier to read and maintain.

- ❖ **Data Classes:** These are classes designed just to hold data. Using the keyword `data` automatically generates standard methods like *equals()*, *hashCode()*, *toString()*, and *copy()*.
- ❖ **Shorter Variable Declaration:** Uses **val** for **immutable (read-only)** variables and **var** for **mutable** variables, and often allows type inference, making declarations cleaner.
- ❖ **Type Inference:** The compiler can often deduce the type of a variable based on the value assigned to it, reducing explicit type declarations.

iii. Interoperability with Java (JVM)

Kotlin was designed to be a "**better Java**" and coexist seamlessly with it.

- ❖ **100% Interoperable:** You can call Kotlin code from Java and Java code from Kotlin in the same project. You can also use all existing Java frameworks and libraries directly in Kotlin.
- ❖ **JVM Target:** Kotlin compiles to Java bytecode, allowing it to run anywhere Java runs (including Android and server environments).

iv. Extension Functions

This feature allows developers to add new functions to an existing class without modifying the original source code or using inheritance.

- ❖ **Purpose:** It helps create cleaner, more readable code by allowing developers to make functions specific to a type (like `String` or `List`) available via dot-notation, making the code feel more idiomatic. For example: *Creating a function fun String.toSnakeCase(): String which can then be called on any string object: "myText".toSnakeCase()*.

v. Coroutines

While a library, Coroutines are a core feature of modern Kotlin development, especially for handling asynchronous tasks.

- ❖ **Simplified Concurrency:** They provide a lightweight alternative to traditional threads, making it much easier and less error-prone to write non-blocking code (like network requests or database access) without freezing the application's UI.
- ❖ **Structured Concurrency:** They ensure that asynchronous operations started in a certain scope are cleaned up when that scope finishes, preventing leaks and simplifying lifecycle management.

vi. Functional Programming Features

Kotlin supports several concepts from functional programming, enabling cleaner, safer data manipulation:

- ❖ **Higher-Order Functions:** Functions can accept other functions as parameters or return them as results.
- ❖ **Lambdas:** Concise, inline functions used frequently with collection operations like map, filter, and forEach.
- ❖ **Immutability Focus:** Kotlin encourages the use of immutable data (**val**), which inherently leads to safer, side-effect-free code.

Question 11: What is the difference between Java and Kotlin in respect to Android development?

Kotlin is more concise, safer, and officially preferred by Google for Android development, while Java remains widely used and supported but involves more boilerplate code and less modern features.

FEATURE	KOTLIN	JAVA
Conciseness	Requires fewer lines of code	More verbose; lots of boilerplate
Null Safety	Built-in null safety with nullable types	Prone to NullPointerException
Interoperability	Fully interoperable with Java	Can call Kotlin code with some setup
Coroutines	Native support for asynchronous programming	Uses threads and callbacks
Extension Functions	Allows adding methods to existing classes	Not supported natively
Smart Casts	Automatic type casting after checks	Requires manual casting
Compilation Speed	Slightly slower in some cases	Generally faster due to maturity
Learning Curve	Easier for beginners due to modern syntax	Familiar to many due to long-standing use
Tooling Support	Excellent in Android Studio	Excellent in Android Studio
Community & Resources	Growing rapidly with official support	Large and mature ecosystem

Question 12: Is there a difference between Java for Desktop and Java for Mobile?

Yes, Java for desktop and Java for mobile differ significantly in terms *of frameworks, runtime environments, and user interface design*, even though they share the same core language.

i. Virtual Machines

- ❖ **Desktop Java:** Utilizes the standard Java Virtual Machine (JVM) for executing bytecode. This allows applications to run on various desktop operating systems (Windows, macOS, Linux) with a compatible JVM.

- ❖ **Mobile Java (Android):** Historically, Android used the **Dalvik Virtual Machine (DVM)**, which was optimized for mobile devices with limited resources. More recently, Android transitioned to the **Android Runtime (ART)**, which offers improved performance. While Android apps are primarily written in Java (or Kotlin), they are compiled into Dalvik Executables (**.dex files**) and run on these specialized virtual machines, not the standard JVM.

ii. Libraries and APIs

- ❖ **Desktop Java:** Accesses the full range of **Java Standard Edition (Java SE)** libraries and potentially **Java Enterprise Edition (Java EE)** libraries for server-side components. This provides a vast array of functionalities for various application types.
- ❖ **Mobile Java (Android):** Relies on the **Android SDK**, which provides a specific set of libraries and APIs tailored for Android devices. While many core Java functionalities are present, some Java SE libraries are not available or are implemented differently in the Android environment.

iii. User Interface (UI) Frameworks

- ❖ **Desktop Java:** Employs UI toolkits like *Swing*, *JavaFX*, or *AWT* for creating graphical user interfaces for desktop applications.
- ❖ **Mobile Java (Android):** Utilizes the Android UI framework, which includes specific components (*Activities*, *Fragments*, *Layouts*, *Views*, etc.) designed for touch-based interaction and mobile device screens.

iv. Development Tools

- ❖ **Desktop Java:** Typically developed using **IDEs** like *Eclipse*, *IntelliJ IDEA*, or *NetBeans*.
- ❖ **Mobile Java (Android):** Primarily developed using **Android Studio**, which is Google's official IDE for Android development and includes the necessary tools and emulators.

v. Targeting Platforms

- ❖ **Desktop Java:** Aims for cross-platform compatibility across various desktop operating systems.
- ❖ **Mobile Java (Android):** Primarily targets the Android operating system. While Java ME (Micro Edition) was historically used for other embedded and mobile devices, Android development is the dominant use of Java in the mobile space today.

In summary, while both utilize the Java programming language, the underlying execution environments, available libraries, and UI frameworks differ significantly between desktop and mobile (Android) Java development, necessitating distinct approaches and tools for each platform.

MODULE 04

INTRODUCTION TO JETPACK COMPOSE

4.1. INTRODUCTION

Jetpack Compose is Android's modern, native UI toolkit designed to simplify and accelerate UI development. It employs a declarative approach, meaning you describe what your UI should look like for a given state, rather than manually manipulating view hierarchies.



Jetpack Compose

Jetpack Compose is Android's recommended modern toolkit for building native UI. It simplifies and accelerates UI development on Android. It quickly brings your app to life with *less code, powerful tools, and intuitive Kotlin APIs*.

So, throughout this **module 4**, we are going to build a simple UI component with declarative functions. You won't be editing any XML layouts or using the Layout Editor. Instead, you will call composable functions to define what elements you want, and the Compose compiler will do the rest.

4.1.1. Composable Functions

Jetpack Compose is built around composable functions. These functions permit us to define our app's UI programmatically by describing how it should look and providing data dependencies, rather than focusing on the process of the UI's construction (*initializing an element, attaching it to a parent, etc.*). To create a composable function, just add the `@Composable` annotation to the function name.

A Compose app is made up of composable functions that is just regular functions marked with `@Composable`, which can call other composable functions. A function is all you need to create a new UI component. The annotation tells Compose to add special support to the function for updating and maintaining your UI over time. Compose lets you structure your code into small chunks. Composable functions are often referred to as "**composables**" for short.

By making small reusable composables, it's easy to build up a library of UI elements used in your app. Each one is responsible for one part of the screen and can be edited independently.

To start a new, Compose project, open Android Studio.

If you're in the Welcome to Android Studio window, click Start a new Android Studio project. If you already have an Android Studio project open, select ***File > New > New Project from the menu bar.***

For a new project, choose Empty Activity from the available templates.

**TO BE
CONTINUED.....**

4.1.2. Layouts

UI elements are hierarchical, with elements contained in other elements. In Compose, you build a UI hierarchy by calling composable functions from other composable functions.

4.1.3. Material Design

Compose is built to support Material Design principles. Many of its UI elements implement Material Design out of the box.

4.1.4. Lists and animations

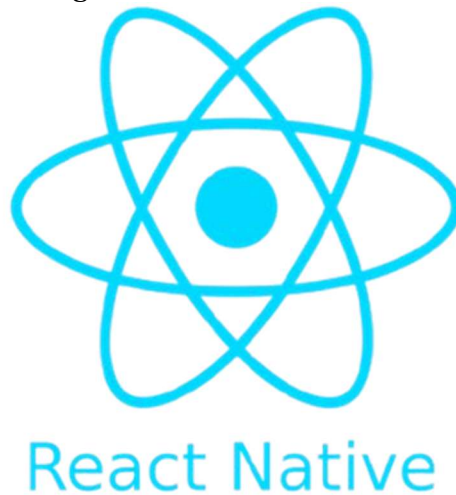
Lists and animations are everywhere in apps.

MODULE 05

INTRODUCTION TO THE REACT NATIVE UI FRAMEWORK

5.1. INTRODUCTION

React Native is an open-source UI software framework created by **Meta (formerly Facebook)** that allows developers to build natively-rendering mobile applications for *iOS, Android, and the web using a single codebase.*



Unlike traditional cross-platform tools that use web views (like **PhoneGap** or **Cordova**), React Native uses actual native UI components, providing the performance and look-and-feel of an app built with **Swift (iOS)** or **Kotlin (Android)**.

React Native was created in **2013** as an internal hackathon project at Facebook and was publicly open-sourced in **March 2015**. **Motivation??** Well.....following Mark Zuckerberg's assessment that relying on HTML5 for the Facebook mobile app was a mistake due to poor performance, engineers sought a better solution.

Engineer Jordan Walke discovered a method to generate native iOS UI elements using JavaScript, which formed the foundation of the new framework. The initial public preview for iOS was showcased at the **React.js Conf** in **January 2015**, and the framework was open-sourced in March of that year.

Support for Android was released later, in **September 2015**, enabling cross-platform development using a single JavaScript codebase. Today, React Native powers major applications like ***Instagram, Facebook, and Microsoft Office***, and development continues with initiatives like the ***"New Architecture"*** to improve performance.

**THIS MODULE OF THE REACT
NATIVE UI FRAMEWORK SHALL
BE CONTINUED IN LEVEL III AS
A COURSE KNOWN AS MOBILE
APPLICATION DEVELOPMENT
THEREFORE, SEE YOU FOR
BTECH NEXT YEAR 🙌**